LFS458 - Kubernetes Administration

Lab Guide

VEGA TRAINING

Table of contents

1. LFS458 - Kubernetes Administration Lab Guide

Welcome to the student lab guide for **LFS458 / CKA preparation**, delivered by **VEGA TRAINING**.

1.1 How to use this guide

Each chapter corresponds to a section of the LFS458 course. Follow the exercises in order - later chapters build on earlier ones.

Work on your assigned lab nodes:

Node	Hostname	IP	Role
Control Plane	`controller`	assigned by instructor	Kubernetes control plane
Worker 1	`worker1`	assigned by instructor	Worker node
Worker 2	`worker2`	assigned by instructor	Worker node

Finding your IPs

Run `ip addr show` on each node, or check `/etc/hosts` if your instructor pre-populated it.

Your username on all nodes is `guru` (password: `work`).

1.2 Lab topology

Component	Value
Kubernetes version	**1.36.1** (upgraded to **1.36.2** in Chapter 4)
Container runtime	**containerd**
CNI	**Calico** (10.244.0.0/16)
OS	Ubuntu 22.04
Control plane endpoint	`controller:6443`

1.3 Quick reference

```
# Check cluster status
kubectl get nodes
kubectl get pods --all-namespaces

# Check component logs
journalctl -u kubelet -f

# Lab files location
ls ~/lfs458/

# Switch context
kubectl config get-contexts
kubectl config use-context <name>
```

Lab files

All pre-staged YAML files for each chapter are in `~/lfs458/chXX-*/`. Simulation test scripts are in `~/chXX_sim.sh`.

Node names

This guide uses `controller`, `worker1`, `worker2` ? not the generic `cp` / `worker` names used in the upstream LFS458 material.

2. Chapter 3: Installation and Configuration

Working directory: `~/lfs458/ch03-install/`

2.1 Exercise 3.1: Install Kubernetes

Run all steps on the CONTROL PLANE node (`controller`) unless stated otherwise.

Prerequisite — lab files must be present

This chapter reads files from `~/lfs458/ch03-install/`. Your instructor stages them with `setup_lab_environment.sh` before class. Verify they exist before you start:

```
ls ~/lfs458/ch03-install/
```

If the directory is missing, run:

```
curl -O https://raw.githubusercontent.com/helmsman-k8s/lfs458-labs/rebuild-2025/setup_lab_environment.sh
sudo bash setup_lab_environment.sh $USER
```

2.1.1 Step 1 — Prepare the system

1. Update the system.

```
sudo apt-get update && sudo apt-get upgrade -y
```

2. Install required packages.

```
sudo apt-get install -y apt-transport-https ca-certificates curl socat tree bash-completion
```

3. Disable swap permanently.

```
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab
```

Verify:

```
free -h | grep Swap
```

```
Swap:          0B        0B        0B
```

4. Load required kernel modules.

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

Make them persist across reboots:

```
cat << EOF | sudo tee /etc/modules-load.d/kubernetes.conf
overlay
br_netfilter
EOF
```

5. Configure kernel networking.

```
cat << EOF | sudo tee /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
```

```
net.ipv4.ip_forward = 1
EOF
```

```
sudo sysctl --system
```

6. Reboot so swap stays off and modules load cleanly.

```
sudo reboot
```

Reconnect via SSH after ~30 seconds, then continue.

2.1.2 Step 2 — Install containerd

7. Add the Docker repository (containerd is distributed here).

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

8. Install containerd.

```
sudo apt-get update && sudo apt-get install -y containerd.io
```

9. Configure containerd to use systemd as the cgroup driver.

```
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Verify it is running:

```
sudo systemctl status containerd --no-pager
```

2.1.3 Step 3 — Install Kubernetes components

10. Add the Kubernetes repository.

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.36/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.36/deb/ /" \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

11. Install kubeadm, kubelet and kubectl at a fixed version.

If apt reports the version is not found, the Debian revision suffix has changed. List what is actually available and use that exact string:

```
sudo apt-cache madison kubeadm
```

```
sudo apt-get update
sudo apt-get install -y kubeadm=1.36.1-1.1 kubelet=1.36.1-1.1 kubectl=1.36.1-1.1 cri-tools
sudo apt-mark hold kubelet kubeadm kubectl
```

12. Configure crictl to use containerd.

```
sudo crictl config \
  --set runtime-endpoint=unix:///run/containerd/containerd.sock \
  --set image-endpoint=unix:///run/containerd/containerd.sock
```

2.1.4 Step 4 — Initialise the cluster

13. Verify that `/etc/hosts` has the correct entries for the cluster hostnames.

```
grep -E 'controller|worker1|worker2' /etc/hosts
```

```
192.168.x.x  controller
192.168.x.x  worker1
192.168.x.x  worker2
```

If these lines are missing, ask your instructor — the lab environment should be pre-configured.

14. Review the kubeadm config file.

```
cat ~/lfs458/ch03-install/kubeadm-config.yaml
```

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: 1.36.1
controlPlaneEndpoint: "controller:6443"
networking:
  podSubnet: 10.244.0.0/16
```

15. Initialise the cluster. This takes 2–3 minutes.

```
sudo cp ~/lfs458/ch03-install/kubeadm-config.yaml /root/
sudo kubeadm init --config=/root/kubeadm-config.yaml \
  --upload-certs --node-name=controller \
  | sudo tee /root/kubeadm-init.out
```

Save the `kubeadm join` command printed at the end — you need it in Exercise 3.2. If the token expires (2 hours) regenerate it with:

```
sudo kubeadm token create --print-join-command
```

16. Set up kubectl access.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

17. Enable kubectl bash completion.

```
source <(kubectl completion bash)
echo "source <(kubectl completion bash)" >> $HOME/.bashrc
```

2.1.5 Step 5 — Install the network plugin (Calico)

18. Install the Calico custom resource definitions. From Calico v3.32 the `projectcalico.org` CRDs ship in their own manifest and must be applied **before** the operator.

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.32.1/manifests/v1_crd_projectcalico_org.yaml
```

```
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org created
...
```

19. Install the Tigera Operator — this manages the Calico lifecycle.

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.32.1/manifests/tigera-operator.yaml
```

```
namespace/tigera-operator created
serviceaccount/tigera-operator created
clusterrole.rbac.authorization.k8s.io/tigera-operator created
clusterrolebinding.rbac.authorization.k8s.io/tigera-operator created
deployment.apps/tigera-operator created
```

Wait for the operator to be ready:

```
kubectl rollout status deployment tigera-operator -n tigera-operator --timeout=120s
```

20. Apply the Calico custom resources. This configures the pod CIDR as `10.244.0.0/16`, which matches `kubeadm-config.yaml` and avoids overlap with the node network.

We use our own file rather than the upstream `custom-resources.yaml`, because upstream defaults to the `192.168.0.0/16` pod CIDR — which would collide with the node network in this lab.

```
kubectl apply -f ~/lfs458/ch03-install/calico-custom-resources.yaml
```

```
installation.operator.tigera.io/default created
apiserver.operator.tigera.io/default created
```

21. Watch Calico come up. This takes 1–2 minutes.

```
watch kubectl get tigerastatus
```

```
NAME        AVAILABLE   PROGRESSING   DEGRADED   SINCE
apiserver   True        False         False      70s
calico      True        False         False      90s
ippools     True        False         False      2m
```

Press `Ctrl+C` when all components show `True` in the `AVAILABLE` column. You can also watch the pods directly:

```
kubectl get pods -n calico-system
```

```
NAME                                       READY   STATUS    RESTARTS   AGE
calico-kube-controllers-57d48bcff9-x4b2k   1/1     Running   0          90s
calico-node-7rqtk                           1/1     Running   0          90s
calico-typha-5f5f4d6b8c-j9p2l              1/1     Running   0          90s
```

Then verify the node is `Ready`:

```
kubectl get nodes
```

```
NAME         STATUS   ROLES           AGE   VERSION
controller   Ready    control-plane   3m    v1.36.1
```

2.2 Exercise 3.2: Grow the Cluster

Open a **new terminal** and SSH into each worker node. Run steps 1–5 on **each worker** (`worker1` then `worker2`).

1. Prepare the system — same as the controller.

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y apt-transport-https ca-certificates curl socat tree bash-completion
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab
sudo modprobe overlay
sudo modprobe br_netfilter
cat << EOF | sudo tee /etc/modules-load.d/kubernetes.conf
overlay
br_netfilter
EOF
cat << EOF | sudo tee /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
```

2. Install containerd.

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update && sudo apt-get install -y containerd.io
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

3. Install Kubernetes components.

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.36/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.36/deb/ /" \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubeadm=1.36.1-1.1 kubelet=1.36.1-1.1 kubectl=1.36.1-1.1 cri-tools
sudo apt-mark hold kubelet kubeadm kubectl
```

4. Configure crictl.

```
sudo crictl config \
  --set runtime-endpoint=unix:///run/containerd/containerd.sock \
  --set image-endpoint=unix:///run/containerd/containerd.sock
```

5. Join the cluster. On the **controller node** first get the join command:

```
sudo kubeadm token create --print-join-command
```

Then run the output on the **worker node**, adding the correct `--node-name`:

```
# On worker1:
sudo kubeadm join controller:6443 --token <your-token> \
  --discovery-token-ca-cert-hash sha256:<your-hash> \
  --node-name=worker1

# On worker2:
sudo kubeadm join controller:6443 --token <your-token> \
  --discovery-token-ca-cert-hash sha256:<your-hash> \
  --node-name=worker2
```

```
This node has joined the cluster.
```

2.3 Exercise 3.3: Finish Cluster Setup

1. Back on the **controller node**, verify all nodes have joined.

```
kubectl get nodes
```

```
NAME         STATUS   ROLES           AGE   VERSION
controller   Ready    control-plane   10m   v1.36.1
worker1      Ready    <none>          2m    v1.36.1
worker2      Ready    <none>          1m    v1.36.1
```

2. Remove the taint from the controller so it can run workloads during training.

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

```
node/controller untainted
error: taint "node-role.kubernetes.io/control-plane" not found   # normal for workers
```

```
kubectl label node worker1 node-role.kubernetes.io/worker=worker
kubectl label node worker2 node-role.kubernetes.io/worker=worker
```

3. Verify all system pods are running.

```
kubectl get pods --all-namespaces
```

All pods should show `Running`. If any are stuck in `Pending` wait another minute and try again.

2.4 Exercise 3.4: Deploy A Simple Application

1. Create an nginx deployment.

```
kubectl create deployment nginx --image=nginx
kubectl get deployments
```

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           8s
```

2. Expose it as a NodePort service.

```
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get svc nginx
```

```
NAME    TYPE       CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.96.x.x     <none>        80:3xxxx/TCP   6s
```

3. Get the assigned NodePort and test with curl.

```
NODE_PORT=$(kubectl get svc nginx -o jsonpath='{.spec.ports[0].nodePort}')
curl http://controller:$NODE_PORT
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

4. Scale the deployment.

```
kubectl scale deployment nginx --replicas=3
kubectl get pods -o wide
```

5. Clean up.

```
kubectl delete deployment nginx
kubectl delete svc nginx
```

2.5 Exercise 3.5: Access from Outside the Cluster

1. Use `exec` to inspect environment variables inside a running pod.

```
kubectl run testpod --image=nginx
kubectl wait --for=condition=Ready pod/testpod --timeout=60s
kubectl exec testpod -- printenv | grep KUBERNETES
```

```
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_HOST=10.96.0.1
```

2. Clean up.

```
kubectl delete pod testpod
```

3. Chapter 4: Kubernetes Architecture

Working directory: `~/lfs458/ch04-architecture/`

3.1 Exercise 4.1: Basic Node Maintenance

In this exercise we will back up the **etcd** database and then upgrade the version of Kubernetes on the control plane and worker nodes.

```
cd ~/lfs458/ch04-architecture/
```

3.1.1 Backup the etcd Database

1. Find the data directory of the **etcd** daemon. All settings for the pod can be found in its manifest.

```
sudo grep data-dir /etc/kubernetes/manifests/etcd.yaml
```

```
    - --data-dir=/var/lib/etcd
```

2. Explore the **etcdctl** options. Use Tab to complete the container name — it has the node name appended to it.

Important: The etcd image is distroless — it contains **no shell**. `kubectl exec ... -- sh` will fail with an executable-not-found error. Run `etcdctl` directly as the command instead.

(a) View the arguments and options available to `etcdctl`.

```
kubectl -n kube-system exec etcd-controller -- etcdctl --help
```

```
NAME:
        etcdctl - A simple command line client for etcd3.

USAGE:
        etcdctl [flags]
...
```

(b) Find the TLS certificate files needed to authenticate with etcd. Since there is no shell in the container, list them from the **node** instead — they are mounted into the container from this path.

```
sudo ls /etc/kubernetes/pki/etcd/
```

```
ca.crt  ca.key  healthcheck-client.crt  healthcheck-client.key
peer.crt  peer.key  server.crt  server.key
```

Why flags, not environment variables: without a shell you cannot use the `ETCDCTL_API=3 ETCDCTL_CACERT=... etcdctl` form, because that syntax is a shell feature. Each call must pass its TLS material as **flags**, as shown below. `etcdctl` v3.4+ already defaults to API v3, so `ETCDCTL_API=3` is no longer needed.

3. Check the health of the etcd database using the loopback IP and port 2379. You do not need to type out the backslashes — they only continue the line.

```
kubectl -n kube-system exec etcd-controller -- etcdctl \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health
```

```
https://127.0.0.1:2379 is healthy: successfully committed proposal: took = 11.942936ms
```

4. Determine how many databases are part of the cluster. Three or five members are common in production for 50%+1 quorum. In our single-node exercise environment you will only see one.

```
kubectl -n kube-system exec etcd-controller -- etcdctl \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --endpoints=https://127.0.0.1:2379 member list
```

```
fb50b7ddbf4930ba, started, controller, https://192.168.2.x:2380,
https://192.168.2.x:2379, false
```

5. View the member list in table format using the `-w table` option.

```
kubectl -n kube-system exec etcd-controller -- etcdctl \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --endpoints=https://127.0.0.1:2379 member list -w table
```

```
+------------------+---------+------+--------------------------+--------------------------+---------+
|        ID        | STATUS  | NAME |        PEER ADDRS        |       CLIENT ADDRS       | LEARNER |
+------------------+---------+------+--------------------------+--------------------------+---------+
| 802d78549985d5a8 | started | controller | https://192.168.2.x:2380  | https://192.168.2.x:2379  | false   |
+------------------+---------+------+--------------------------+--------------------------+---------+
```

6. Back up the etcd database using the `snapshot` argument. The snapshot is saved inside the container's data directory `/var/lib/etcd/`.

```
kubectl -n kube-system exec etcd-controller -- etcdctl \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --endpoints=https://127.0.0.1:2379 snapshot save /var/lib/etcd/snapshot.db
```

```
{"level":"info",...,"msg":"created temporary db file","path":"/var/lib/etcd/snapshot.db.part"}
{"level":"info",...,"msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info",...,"msg":"saved","path":"/var/lib/etcd/snapshot.db"}
Snapshot saved at /var/lib/etcd/snapshot.db
```

7. Verify the snapshot file exists from the node perspective.

```
sudo ls -l /var/lib/etcd/
```

```
total 3888
drwx------ 4 root root    4096 Aug 25 11:22 member
-rw------- 1 root root 3973152 Aug 25 18:42 snapshot.db
```

8. Back up the snapshot and cluster config files locally in case the node becomes unavailable.

```
mkdir $HOME/backup
sudo cp /var/lib/etcd/snapshot.db $HOME/backup/snapshot.db-$(date +%m-%d-%y)
sudo cp /root/kubeadm-config.yaml $HOME/backup/
sudo cp -r /etc/kubernetes/pki/etcd $HOME/backup/
```

Any mistakes during restore may render the cluster unusable. Attempt a restore only after the final lab exercise. More on the restore process: https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/#restoring-an-etcd-cluster

---

3.1.2 Upgrade the Cluster — Control Plane Node

1. Update the package metadata for APT.

```
sudo apt update
```

```
...
```

2. In this exercise we perform a **patch upgrade** (1.36.1 → 1.36.2), staying within the same minor release. Because the apt repository is already pinned to `v1.36`, no repository change is required — simply refresh the package metadata.

```
sudo apt-get update
```

Note: For a **minor** upgrade (for example 1.36 → 1.37) you would first have to repoint the repository, because each minor release is published in its own repository:

```
sudo sed -i 's/v1.36/v1.37/g' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
```

Kubernetes only supports upgrading **one minor version at a time**.

3. View the available kubeadm packages to confirm the target patch version is available.

```
sudo apt-cache madison kubeadm
```

```
kubeadm | 1.36.2-2.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
kubeadm | 1.36.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
kubeadm | 1.36.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
...
```

4. Remove the hold on **kubeadm** and upgrade it to the target patch release.

```
sudo apt-mark unhold kubeadm
```

```
Canceled hold on kubeadm.
```

```
sudo apt-get install -y kubeadm=1.36.2-2.1
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
...
```

5. Hold the package again to prevent unintended upgrades.

```
sudo apt-mark hold kubeadm
```

```
kubeadm set on hold.
```

6. Verify the new kubeadm version.

```
sudo kubeadm version
```

```
kubeadm version: &version.Info{Major:"1", Minor:"36", GitVersion:"v1.36.2", ...}
```

7. Drain the CP node to evict as many pods as possible before upgrading. DaemonSet pods such as Calico must remain.

```
kubectl drain controller --ignore-daemonsets
```

```
node/controller cordoned
Warning: ignoring DaemonSet-managed Pods: calico-system/calico-node-5tv9d, kube-system/kube-proxy-8x9c5
evicting pod kube-system/coredns-5d78c9869d-z5ngb
evicting pod calico-system/calico-kube-controllers-788c7d7585-wnb5b
evicting pod kube-system/coredns-5d78c9869d-4h2bs
pod/calico-kube-controllers-788c7d7585-wnb5b evicted
pod/coredns-5d78c9869d-z5ngb evicted
pod/coredns-5d78c9869d-4h2bs evicted
node/controller drained
```

Note: If the drain loops with `Cannot evict pod as it would violate the pod's disruption budget` errors, wait — it will resolve automatically once a replacement pod starts on a worker node (typically 2–3 minutes).

8. Review the upgrade plan. Read through the output to understand what will change.

```
sudo kubeadm upgrade plan
```

```
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the cluster...
[upgrade] Running cluster health checks
[upgrade/versions] Cluster version: v1.36.1
[upgrade/versions] kubeadm version: v1.36.2
[upgrade/versions] Target version: v1.36.2
[upgrade/versions] Latest version in the v1.36 series: v1.36.2
...
```

9. Apply the upgrade. Answer **y** when prompted. This will take several minutes.

```
sudo kubeadm upgrade apply v1.36.2
```

```
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the cluster...
[upgrade/version] You have chosen to change the cluster version to "v1.36.2"
[upgrade/versions] Cluster version: v1.36.1
[upgrade/versions] kubeadm version: v1.36.2
[upgrade] Are you sure you want to proceed? [y/N]: y
[upgrade/prepull] Pulling images required for setting up a Kubernetes cluster
...
```

10. Check node status. The CP should show `Ready,SchedulingDisabled` and still show the old version until kubelet is upgraded.

```
kubectl get node
```

```
NAME     STATUS                     ROLES           AGE    VERSION
controller   Ready,SchedulingDisabled   control-plane   109m   v1.36.1
worker1      Ready                      <none>          61m    v1.36.1
```

11. Release the hold on **kubelet** and **kubectl**.

```
sudo apt-mark unhold kubelet kubectl
```

```
Canceled hold on kubelet.
Canceled hold on kubectl.
```

12. Upgrade both packages to match kubeadm.

```
sudo apt-get install -y kubelet=1.36.2-2.1 kubectl=1.36.2-2.1
```

```
Reading package lists... Done
...
```

13. Hold the packages again.

```
sudo apt-mark hold kubelet kubectl
```

```
kubelet set on hold.
kubectl set on hold.
```

14. Restart the daemons.

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

15. Verify the CP node now shows the new version. The worker still shows the old version — we will upgrade it next.

```
kubectl get node
```

```
NAME     STATUS                     ROLES           AGE    VERSION
controller   Ready,SchedulingDisabled   control-plane   113m   v1.36.2
worker1      Ready                      <none>          65m    v1.36.1
```

16. Make the CP available for scheduling again.

```
kubectl uncordon controller
```

```
node/controller uncordoned
```

17. Verify the CP now shows `Ready` status.

```
kubectl get node
```

```
NAME     STATUS   ROLES           AGE    VERSION
controller   Ready    control-plane   114m   v1.36.2
worker1      Ready    <none>          66m    v1.36.1
```

3.1.3 Upgrade the Cluster — Worker Node

Open a **second terminal** connected to the **worker node** for steps 18–27. You will also need the CP terminal open.

18. On the **worker node**, remove the hold on kubeadm.

```
sudo apt-mark unhold kubeadm
```

```
Canceled hold on kubeadm.
```

19. Refresh the package metadata. As on the control plane, the repository is already pinned to `v1.36`, so no repository change is needed for a patch upgrade.

```
sudo apt-get update
```

20. View available packages to confirm the target patch version.

```
sudo apt-cache madison kubeadm
```

```
kubeadm | 1.36.2-2.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
kubeadm | 1.36.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
kubeadm | 1.36.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb  Packages
...
```

21. Install the updated kubeadm on the worker.

```
sudo apt-get update && sudo apt-get install -y kubeadm=1.36.2-2.1
```

```
...
Setting up kubeadm (1.36.2-2.1) ...
```

22. Hold kubeadm again.

```
sudo apt-mark hold kubeadm
```

```
kubeadm set on hold.
```

23. Back on the **CP node**, drain the worker node to evict its pods before upgrading.

```
kubectl drain worker1 --ignore-daemonsets
```

```
node/worker1 cordoned
Warning: ignoring DaemonSet-managed Pods: calico-system/calico-node-gzdk6, kube-system/kube-proxy-lpsmq
evicting pod kube-system/coredns-5d78c9869d-h4p7v
evicting pod kube-system/coredns-5d78c9869d-d4nv8
pod/coredns-5d78c9869d-h4p7v evicted
pod/coredns-5d78c9869d-d4nv8 evicted
node/worker1 drained
```

24. Back on the **worker node**, download the updated node configuration.

```
sudo kubeadm upgrade node
```

```
[upgrade] Reading configuration from the cluster...
[preflight] Running pre-flight checks
[preflight] Skipping prepull. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
```

```
[upgrade] The configuration for this node was successfully updated!
[upgrade] Now you should go ahead and upgrade the kubelet package using your package manager.
```

25. Remove the hold on kubelet and kubectl, then upgrade them.

```
sudo apt-mark unhold kubelet kubectl
```

```
Canceled hold on kubelet.
Canceled hold on kubectl.
```

```
sudo apt-get install -y kubelet=1.36.2-2.1 kubectl=1.36.2-2.1
```

```
Reading package lists... Done
...
Setting up kubectl (1.36.2-2.1) ...
Setting up kubelet (1.36.2-2.1) ...
```

26. Hold the packages again.

```
sudo apt-mark hold kubelet kubectl
```

```
kubelet set on hold.
kubectl set on hold.
```

27. Restart the daemons on the worker node.

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

28. Back on the **CP node**, check the status of both nodes.

```
kubectl get node
```

```
NAME         STATUS                     ROLES           AGE    VERSION
controller   Ready                      control-plane   118m   v1.36.2
worker1      Ready,SchedulingDisabled   <none>          70m    v1.36.2
worker2      Ready                      <none>          68m    v1.36.1
```

29. Uncordon the worker node to allow pods to be scheduled on it again.

```
kubectl uncordon worker1
```

```
node/worker1 uncordoned
```

30. Verify both nodes show `Ready` and the same upgraded version.

```
kubectl get nodes
```

```
NAME         STATUS   ROLES           AGE    VERSION
controller   Ready    control-plane   119m   v1.36.2
worker1      Ready    <none>          71m    v1.36.2
worker2      Ready    <none>          69m    v1.36.1
```

31. Repeat steps 18–30 for **worker2**. The procedure is identical — SSH into worker2 and follow the same upgrade sequence.

Once complete, verify all three nodes are on the same version:

```
kubectl get nodes
```

```
NAME         STATUS   ROLES           AGE    VERSION
controller   Ready    control-plane   125m   v1.36.2
worker1      Ready    <none>          77m    v1.36.2
worker2      Ready    <none>          75m    v1.36.2
```

3.2 Exercise 4.2: Working with CPU and Memory Constraints

We will deploy a **stress** application inside a container and use `resource limits` to constrain what it can consume.

```
cd ~/lfs458/ch04-architecture/
```

1. Create a deployment called `hog` using the `vish/stress` container image and verify it is running.

```
kubectl create deployment hog --image vish/stress
```

```
deployment.apps/hog created
```

```
kubectl get deployments
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
hog    1/1     1            1           13s
```

2. Describe the deployment and view the output in YAML format. Note there are no resource limits set — the `resources:` field shows empty curly brackets `{}`.

```
kubectl describe deployment hog
```

```
Name:       hog
Namespace:  default
Labels:     app=hog
Annotations: deployment.kubernetes.io/revision: 1
...
```

```
kubectl get deployment hog -o yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
...
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: hog
    spec:
      containers:
      - image: vish/stress
        imagePullPolicy: Always
        name: stress
        resources: {}
...
```

3. Save the deployment YAML to a file.

```
kubectl get deployment hog -o yaml > hog.yaml
```

4. Edit the file to remove the `status:` section, `creationTimestamp`, and other generated fields. Then add memory resource limits as shown below. Find the `resources: {}` line and replace it with the limits block.

```
vim hog.yaml
```

```
....
        name: stress
        resources:              # Edit to remove {}
          limits:               # Add these 4 lines
            memory: "4Gi"
          requests:
            memory: "2500Mi"
        terminationMessagePath: /dev/termination-log
....
```

5. Replace the deployment using the edited file.

```
kubectl replace -f hog.yaml
```

```
deployment.apps/hog replaced
```

Note: If you see `the object has been modified; please apply your changes to the latest version`, the deployment controller updated the object between your `kubectl get` and `kubectl replace`. Re-run the `kubectl get deployment hog -o yaml > hog.yaml` command and reapply your edits.

6. Verify the deployment now shows the memory resource limits.

```
kubectl get deployment hog -o yaml
```

```
....
        resources:
          limits:
            memory: 4Gi
          requests:
            memory: 2500Mi
        terminationMessagePath: /dev/termination-log
....
```

7. View the pod name and check its logs to see the stress application output.

```
kubectl get po
```

```
NAME                   READY   STATUS    RESTARTS   AGE
hog-64cbfcc7cf-lwq66   1/1     Running   0          2m
```

```
kubectl logs <hog-pod-name>
```

```
I1102 16:16:42.638972  1 main.go:26] Allocating "0" memory, in
  "4Ki" chunks, with a 1ms sleep between allocations
I1102 16:16:42.639064  1 main.go:29] Allocated "0" memory
```

8. Open a second and third terminal to access both CP and worker nodes. Run `top` on each to monitor resource usage. The **stress** command is not consuming resources yet since no `args:` have been set.

9. Edit `hog.yaml` to add CPU and memory consumption arguments for the **stress** container. The `args:` entry should be indented at the same level as `resources:`. If you get stuck, the completed file is at `~/lfs458/ch04-architecture/solutions/hog-with-resources.yaml`.

```
vim hog.yaml
```

```
....
        resources:
          limits:
            cpu: "1"
            memory: "4Gi"
          requests:
            cpu: "0.5"
            memory: "500Mi"
        args:
        - -cpus
        - "2"
        - -mem-total
        - "950Mi"
        - -mem-alloc-size
        - "100Mi"
        - -mem-alloc-sleep
        - "1s"
....
```

10. Delete and recreate the deployment. Watch the `top` output on both nodes — you should see CPU usage spike and memory allocated in 100Mi chunks.

```
kubectl delete deployment hog
```

```
deployment.apps "hog" deleted
```

```
kubectl create -f hog.yaml
```

```
deployment.apps/hog created
```

If `top` does not show high usage, check the pod logs for errors. A common mistake is using `Mi` (case sensitive) — `mi` will cause the container to panic.

```
kubectl get pod
kubectl logs <hog-pod-name>
```

3.3 Exercise 4.3: Resource Limits for a Namespace

The previous exercise set limits on a specific deployment. You can also set limits on an entire **namespace** using a `LimitRange` object. When set, the `hog` deployment should not be able to exceed those namespace-level limits.

```
cd ~/lfs458/ch04-architecture/
```

1. Create a new namespace called `low-usage-limit` and verify it exists.

```
kubectl create namespace low-usage-limit
```

```
namespace/low-usage-limit created
```

```
kubectl get namespace
```

```
NAME              STATUS   AGE
default           Active   1h
kube-node-lease   Active   1h
kube-public       Active   1h
kube-system       Active   1h
low-usage-limit   Active   42s
```

2. The `low-resource-range.yaml` file is already staged in your lab folder. Review it.

```
cat ~/lfs458/ch04-architecture/low-resource-range.yaml
```

```
apiVersion: v1
kind: LimitRange
metadata:
  name: low-resource-range
spec:
  limits:
  - default:
      cpu: 1
      memory: 500Mi
    defaultRequest:
      cpu: 0.5
      memory: 100Mi
    type: Container
```

3. Apply the LimitRange to the `low-usage-limit` namespace.

```
kubectl create -f ~/lfs458/ch04-architecture/low-resource-range.yaml \
  -n low-usage-limit
```

```
limitrange/low-resource-range created
```

4. Verify the LimitRange was created. Note it only exists in the `low-usage-limit` namespace — the default namespace shows nothing.

```
kubectl get LimitRange
```

```
No resources found in default namespace.
```

```
kubectl get LimitRange --all-namespaces
```

```
NAMESPACE         NAME                CREATED AT
low-usage-limit   low-resource-range  2024-06-23T10:23:57Z
```

5. Create a new deployment in the `low-usage-limit` namespace.

```
kubectl -n low-usage-limit \
  create deployment limited-hog --image vish/stress
```

```
deployment.apps/limited-hog created
```

6. List deployments across all namespaces. Note that `hog` continues to run in the `default` namespace.

```
kubectl get deployments --all-namespaces
```

```
NAMESPACE         NAME              READY   UP-TO-DATE   AVAILABLE   AGE
default           hog               1/1     1            1           7m57s
calico-system     calico-kube-controllers   1/1     1            1           2d10h
kube-system       coredns           2/2     2            2           2d10h
low-usage-limit   limited-hog       1/1     1            1           9s
```

7. View the pods in the `low-usage-limit` namespace.

```
kubectl -n low-usage-limit get pods
```

```
NAME                           READY   STATUS    RESTARTS   AGE
limited-hog-2556092078-wnpnv   1/1     Running   0          2m11s
```

8. Look at the pod details. Notice it has inherited the resource limits from the namespace LimitRange.

```
kubectl -n low-usage-limit \
  get pod <limited-hog-pod-name> -o yaml
```

```
...
spec:
  containers:
  - image: vish/stress
    imagePullPolicy: Always
    name: stress
    resources:
      limits:
        cpu: "1"
        memory: 500Mi
      requests:
        cpu: 500m
        memory: 100Mi
    terminationMessagePath: /dev/termination-log
...
```

9. Copy `hog.yaml` to a new file and add a `namespace:` line so the deployment runs in `low-usage-limit`. Delete the `selfLink:` line if it exists.

```
cp hog.yaml hog2.yaml
vim hog2.yaml
```

```
....
  labels:
    app: hog
  name: hog
  namespace: low-usage-limit    #<-- Add this line, delete selfLink below if present
spec:
....
```

10. Create the deployment in the `low-usage-limit` namespace.

```
kubectl create -f hog2.yaml
```

```
deployment.apps/hog created
```

11. List all deployments. You should see two deployments with the same name `hog` — one in `default` and one in `low-usage-limit`.

```
kubectl get deployments --all-namespaces
```

```
NAMESPACE         NAME              READY   UP-TO-DATE   AVAILABLE   AGE
default           hog               1/1     1            1           24m
calico-system     calico-kube-controllers   1/1     0            0           4h
kube-system       coredns           2/2     2            2           4h
```

```
low-usage-limit   hog               1/1     1            1           26s
low-usage-limit   limited-hog       1/1     1            1           5m11s
```

12. Check `top` on both node terminals. Both `hog` deployments should be consuming about the same resources — the per-deployment settings from `hog.yaml` override the global namespace limits.

```
25128 root   20   0  958532 954672  3180 R 100.0 11.7   0:52.27 stress
24875 root   20   0  958532 954800  3180 R 100.3 11.7  41:04.97 stress
```

13. Delete all `hog` deployments to recover cluster resources.

```
kubectl -n low-usage-limit delete deployment hog limited-hog
```

```
deployment.apps "hog" deleted
deployment.apps "limited-hog" deleted
```

```
kubectl delete deployment hog
```

```
deployment.apps "hog" deleted
```

4. Chapter 5: APIs and Access

Working directory: `~/lfs458/ch05-api-access/`

4.1 Exercise 5.1: Configuring TLS Access

Using the Kubernetes API, **kubectl** makes API calls on your behalf. With the appropriate TLS keys you can also use **curl** directly to interact with the API server. API requests must pass information as JSON — kubectl converts `.yaml` to JSON automatically.

```
cd ~/lfs458/ch05-api-access/
```

1. Review the kubectl configuration file. We will use the three certificates and the API server address from this file.

```
less $HOME/.kube/config
```

You will see the cluster's server address, and three base64-encoded certificates: `certificate-authority-data`, `client-certificate-data`, and `client-key-data`. Press `q` to exit.

2. Create variables from the certificate information in the config file. Begin with the `client-certificate-data` key.

```
export client=$(grep client-cert $HOME/.kube/config | cut -d" " -f 6)
echo $client
```

You will see a long base64 string — this is your cluster's client certificate. The value will be unique to your cluster.

3. Collect the `client-key-data` as the `key` variable.

```
export key=$(grep client-key-data $HOME/.kube/config | cut -d" " -f 6)
echo $key
```

You will see another long base64 string — your client private key.

4. Set the `auth` variable with the `certificate-authority-data` key.

```
export auth=$(grep certificate-authority-data $HOME/.kube/config | cut -d" " -f 6)
echo $auth
```

You will see another long base64 string — the cluster's Certificate Authority (CA).

5. Decode and save the three keys as PEM files for use with `curl`.

```
echo $client | base64 -d - > ./client.pem
echo $key | base64 -d - > ./client-key.pem
echo $auth | base64 -d - > ./ca.pem
```

6. Pull the API server URL from the config file.

```
kubectl config view | grep server
```

```
server: https://controller:6443
```

7. Use `curl` with the PEM files to connect to the API server and list pods. Use the server address from the previous step.

```
curl --cert ./client.pem \
  --key ./client-key.pem \
  --cacert ./ca.pem \
  https://controller:6443/api/v1/pods
```

```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "239414"
```

```
  },
  "items": [
    ...all running pods listed as JSON objects...
  ]
}
```

8. Now use `curl` to create a new pod using a JSON file. The `curlpod.json` file is already staged in your lab folder. Review it first.

```
cat ~/lfs458/ch05-api-access/curlpod.json
```

```
{
    "kind": "Pod",
    "apiVersion": "v1",
    "metadata": {
        "name": "curlpod",
        "namespace": "default",
        "labels": {
            "name": "examplepod"
        }
    },
    "spec": {
        "containers": [{
            "name": "nginx",
            "image": "nginx",
            "ports": [{"containerPort": 80}]
        }]
    }
}
```

9. Use `curl` with an XPOST call to create the pod from the JSON file. There will be a lot of output — the last few lines should show a `Pending` phase as the pod begins creation.

```
curl --cert ./client.pem \
  --key ./client-key.pem \
  --cacert ./ca.pem \
  https://controller:6443/api/v1/namespaces/default/pods \
  -XPOST -H'Content-Type: application/json' \
  -d@curlpod.json
```

```
{
  "kind": "Pod",
  "apiVersion": "v1",
  "metadata": {
    "name": "curlpod",
    ...
  },
  "status": {
    "phase": "Pending"
  }
}
```

10. Verify the new pod exists and is running.

```
kubectl get pods
```

```
NAME       READY   STATUS    RESTARTS   AGE
curlpod    1/1     Running   0          45s
```

4.2 Exercise 5.2: Explore API Calls

1. Use **strace** to observe what API calls `kubectl` makes under the hood. Install strace if not present, then check current endpoints.

```
sudo apt-get install -y strace
kubectl get endpoints
```

```
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME         ENDPOINTS           AGE
kubernetes   192.168.2.x:6443    3h
```

Note: The deprecation warning is expected on Kubernetes 1.33+ and can be ignored.

2. Run the same command prefixed with `strace`. Near the end of the output you will see several `openat` calls referencing the local discovery cache directory. Redirect to a file and grep if the output is too large.

```
strace kubectl get endpoints
```

```
execve("/usr/bin/kubectl", ["kubectl", "get", "endpoints"], [/*....
....
openat(AT_FDCWD, "/home/<your-user>/.kube/cache/discovery/controller_6443/...
... many system calls ...
```

3. Change to the cache discovery directory and explore its contents.

```
cd $HOME/.kube/cache/discovery/
ls
```

```
controller_6443
```

```
cd controller_6443/
ls
```

```
admissionregistration.k8s.io   certificates.k8s.io   node.k8s.io
apiextensions.k8s.io            coordination.k8s.io   policy
apiregistration.k8s.io          crd.projectcalico.org  rbac.authorization.k8s.io
apps                            discovery.k8s.io      scheduling.k8s.io
authentication.k8s.io           events.k8s.io         servergroups.json
authorization.k8s.io            extensions            storage.k8s.io
autoscaling                     flowcontrol.apiserver.k8s.io   v1
batch                           networking.k8s.io
```

4. List the sub-files recursively.

```
find .
```

```
.
./storage.k8s.io
./storage.k8s.io/v1beta1
./storage.k8s.io/v1beta1/serverresources.json
./storage.k8s.io/v1
./storage.k8s.io/v1/serverresources.json
./rbac.authorization.k8s.io
... (one entry per API group)
```

5. View the objects available in version 1 of the API. For each object you can see the verbs (actions) available.

```
python3 -m json.tool v1/serverresources.json
```

```
{
    "apiVersion": "v1",
    "groupVersion": "v1",
    "kind": "APIResourceList",
    "resources": [
        {
            "kind": "Binding",
            "name": "bindings",
            "namespaced": true,
            "singularName": "",
            "verbs": [
                "create"
            ]
        },
...
```

6. Pipe through `less` to navigate the output more easily.

```
python3 -m json.tool v1/serverresources.json | less
```

7. Find the `shortNames` for resources — these are the abbreviated names you can use with `kubectl`. Look for the endpoint shortname.

```
python3 -m json.tool v1/serverresources.json | grep -A 4 shortNames
```

```
"shortNames": [
    "ep"
],
```

8. Use the shortName to verify it works.

```
kubectl get ep
```

```
NAME         ENDPOINTS           AGE
kubernetes   10.96.0.1:6443      3h
```

9. Count how many objects are in the v1 API group.

```
python3 -m json.tool v1/serverresources.json | grep kind
```

```
    "kind": "APIResourceList",
            "kind": "Binding",
            "kind": "ComponentStatus",
            "kind": "ConfigMap",
            "kind": "Endpoints",
            "kind": "Event",
... (many more resource kinds)
```

10. Look at the `apps/v1` API group — it contains nine more resource types.

```
python3 -m json.tool apps/v1/serverresources.json | grep kind
```

```
    "kind": "APIResourceList",
            "kind": "ControllerRevision",
            "kind": "DaemonSet",
            "kind": "DaemonSet",
            "kind": "Deployment",
... (more resource kinds)
```

11. Return to the home directory and delete the `curlpod` to recoup system resources.

```
cd $HOME
kubectl delete po curlpod
```

```
pod "curlpod" deleted
```

12. Take a look at other files in the cache directory as time permits to understand the full API surface.

5. Chapter 6: API Objects

5.1 Overview

In this lab you will work with:

- Jobs (one-shot, parallel, deadline-bounded)
- CronJobs (scheduled Jobs)
- ConfigMaps and Secrets as API objects

The YAML files are in `~/lfs458/ch06-api-objects/`. Chapters that require editing a file have a completed version in the `solutions/` subfolder.

5.2 Lab 6.1 — Jobs

5.2.1 Create a basic Job

```
cp ~/lfs458/ch06-api-objects/job.yaml ~/job.yaml
cat ~/job.yaml
```

Apply it and watch it complete:

```
kubectl apply -f ~/job.yaml
kubectl get jobs -w
```

Once complete, check the pod:

```
JOB_POD=$(kubectl get pod -l job-name=sleepy -o jsonpath='{.items[0].metadata.name}')
kubectl logs $JOB_POD
kubectl describe job sleepy
```

5.2.2 Add completions

Edit `~/job.yaml` to run the job **5 times total** by setting `spec.completions: 5`. The solution is at `~/lfs458/ch06-api-objects/solutions/job-completions.yaml`.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
kubectl get jobs -w
```

5.2.3 Add parallelism

Now also set `spec.parallelism: 2` so 2 pods run at a time. The solution is at `~/lfs458/ch06-api-objects/solutions/job-parallelism.yaml`.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
kubectl get pods -w   # You should see 2 pods running simultaneously
```

5.2.4 Add an active deadline

Set `spec.activeDeadlineSeconds: 15` so the job is killed if it hasn't finished in 15 seconds. The solution is at `~/lfs458/ch06-api-objects/solutions/job-deadline.yaml`.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
watch kubectl get jobs
```

Observe the job being terminated early with reason `DeadlineExceeded`.

5.2.5 Clean up

```
kubectl delete job sleepy 2>/dev/null; true
```

5.3 Lab 6.2 — CronJobs

```
cp ~/lfs458/ch06-api-objects/cronjob.yaml ~/cronjob.yaml
cat ~/cronjob.yaml
```

Apply and observe:

```
kubectl apply -f ~/cronjob.yaml
kubectl get cronjobs -w
```

The CronJob runs every minute. Wait for the first execution:

```
kubectl get jobs --watch
```

Once a Job appears, check its pod's logs:

```
CRON_POD=$(kubectl get pod -l app=date-job --sort-by=.metadata.creationTimestamp \
  -o jsonpath='{.items[-1].metadata.name}')
kubectl logs $CRON_POD
```

5.3.1 Add a job deadline

Edit `~/cronjob.yaml` to add `spec.jobTemplate.spec.activeDeadlineSeconds: 10` and change the sleep duration so the job exceeds the deadline. The solution is at `~/lfs458/ch06-api-objects/solutions/cronjob-deadline.yaml`.

```
kubectl delete cronjob date-job
kubectl apply -f ~/cronjob.yaml
kubectl get jobs -w
```

5.3.2 Clean up

```
kubectl delete cronjob date-job 2>/dev/null; true
```

5.4 Lab 6.3 — ConfigMaps

Create a ConfigMap from literal values:

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info

kubectl describe configmap app-config
kubectl get configmap app-config -o yaml
```

Use the ConfigMap in a pod as environment variables:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: config-test
spec:
  containers:
  - name: app
    image: nginx
    envFrom:
    - configMapRef:
```

```
        name: app-config
EOF
```

Verify the env vars are present:

```
kubectl exec config-test -- env | grep -E "APP_ENV|LOG_LEVEL"
```

Clean up:

```
kubectl delete pod config-test
kubectl delete configmap app-config
```

5.5 Lab 6.4 — Secrets

Create a Secret:

```
kubectl create secret generic db-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=s3cr3t!

kubectl get secret db-secret -o yaml
```

Notice the `data` values are base64-encoded (not encrypted at rest by default). Decode one:

```
kubectl get secret db-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
echo   # newline
```

Clean up:

```
kubectl delete secret db-secret
```

6. Chapter 7: Deployments, ReplicaSets, and DaemonSets

6.1 Overview

In this lab you will:

- Work with ReplicaSets directly
- Create and manage Deployments (scale, rollout, rollback)
- Create a DaemonSet with `OnDelete` and `RollingUpdate` strategies

Lab files are in `~/lfs458/ch07-deployments/`. The `solutions/` subfolder contains completed manifests for files you need to edit.

6.2 Lab 7.1 — ReplicaSets

Create a ReplicaSet:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: webserver
  labels:
    app: webserver
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      labels:
        app: webserver
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
EOF
```

List pods managed by the ReplicaSet:

```
kubectl get rs webserver
kubectl get pods -l app=webserver
```

Delete one pod and watch the ReplicaSet recreate it:

```
RS_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $RS_POD
kubectl get pods -l app=webserver -w
```

6.2.1 Clean up

```
kubectl delete rs webserver
```

6.3 Lab 7.2 — Deployments

Create a Deployment:

```
kubectl create deployment webserver --image=nginx:1.25 --replicas=3
kubectl rollout status deployment webserver
kubectl get pods -l app=webserver
```

Inspect the Deployment:

```
WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

6.3.1 Scale the Deployment

```
kubectl scale deployment webserver --replicas=5
kubectl get pods -l app=webserver -w
```

6.3.2 Update the image (rolling update)

```
kubectl set image deployment/webserver nginx=nginx:1.26
```

Watch the update progress — old pods are terminated one by one as new ones become ready. Press `Ctrl+C` once the pod count settles:

```
kubectl get pods -l app=webserver -w
```

Confirm the rollout has finished:

```
kubectl rollout status deployment webserver
```

```
deployment "webserver" successfully rolled out
```

Verify the new image is running:

```
WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

6.3.3 Rollback

```
kubectl rollout history deployment webserver
kubectl rollout undo deployment webserver
kubectl rollout status deployment webserver

WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

The image should be back to `nginx:1.25`.

6.3.4 Pause and resume a rollout

```
# Start an update and immediately pause
kubectl set image deployment/webserver nginx=nginx:1.27
kubectl rollout pause deployment webserver

# Inspect — only some pods updated
kubectl get pods -l app=webserver

# Resume
kubectl rollout resume deployment webserver
kubectl rollout status deployment webserver
```

6.3.5 Clean up

```
kubectl delete deployment webserver
```

6.4 Lab 7.3 — DaemonSets

Create a DaemonSet with the `OnDelete` update strategy so you control when each node's pod is updated:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ds-one
  labels:
    app: ds-one
spec:
  updateStrategy:
    type: OnDelete
  selector:
    matchLabels:
      system: ds-one
  template:
    metadata:
      labels:
        system: ds-one
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
EOF
```

Verify the DaemonSet runs one pod per node:

```
kubectl get ds ds-one
kubectl get pods -l system=ds-one -o wide
```

Update the image:

```
kubectl set image ds/ds-one nginx=nginx:1.26
```

Note: `rollout status` is not supported for `OnDelete` DaemonSets — pods are only updated when you delete them manually.

With `OnDelete`, pods are only updated when you delete them manually:

```
DS_POD=$(kubectl get pod -l system=ds-one -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $DS_POD
# A new pod is created with the updated image
kubectl get pods -l system=ds-one -o wide
```

Verify the updated pod runs the new image:

```
DS_POD=$(kubectl get pod -l system=ds-one -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $DS_POD | grep Image:
```

6.4.1 Switch to RollingUpdate

Edit the DaemonSet in place:

```
kubectl patch ds ds-one -p '{"spec":{"updateStrategy":{"type":"RollingUpdate"}}}'
kubectl get ds ds-one -o jsonpath='{.spec.updateStrategy.type}'
```

Now update the image — all pods roll over automatically:

```
kubectl set image ds/ds-one nginx=nginx:1.27
kubectl rollout status ds/ds-one
```

6.4.2 Clean up

```
kubectl delete ds ds-one
```

7. Chapter 8: Helm and Kustomize

7.1 Overview

In this lab you will:

- Install and use Helm to deploy an application from a chart
- Install `metrics-server` with Helm and configure it for insecure TLS (lab environment)
- Configure Horizontal Pod Autoscaling (HPA) and generate load with `siege`
- Use Kustomize to manage environment-specific configuration overlays

---

7.2 Lab 8.1 — Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

Add a repository and search it:

```
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

helm search repo bitnami/nginx
```

7.3 Lab 8.2 — Deploy an Application with Helm

Install an nginx chart into its own namespace:

```
kubectl create namespace helm-nginx
helm install my-nginx bitnami/nginx --namespace helm-nginx
```

Watch the pods come up:

```
kubectl get pods -n helm-nginx -w
```

Inspect what Helm created:

```
helm list -n helm-nginx
helm status my-nginx -n helm-nginx
kubectl get all -n helm-nginx
```

Clean up:

```
helm uninstall my-nginx -n helm-nginx
kubectl delete namespace helm-nginx
```

7.4 Lab 8.3 — Install metrics-server

`metrics-server` is required for `kubectl top` and for HPA to function. In a lab environment, the kubelets use self-signed certificates, so you must pass `--kubelet-insecure-tls`.

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update

helm upgrade --install metrics-server metrics-server/metrics-server \
```

```
  --namespace kube-system \
  --set args="{--kubelet-insecure-tls}"
```

Wait for it to be ready:

```
kubectl rollout status deployment metrics-server -n kube-system --timeout=120s
```

Verify:

```
kubectl top nodes
kubectl top pods -A
```

7.5 Lab 8.4 — Horizontal Pod Autoscaler

7.5.1 Deploy a CPU-hungry application

```
kubectl create deployment php-apache \
  --image=registry.k8s.io/hpa-example
kubectl set resources deployment php-apache --requests=cpu=200m

kubectl expose deployment php-apache --port=80
```

Create the HPA:

```
kubectl autoscale deployment php-apache \
  --cpu=50% \
  --min=1 \
  --max=10

kubectl get hpa php-apache -w
```

7.5.2 Generate load with siege

```
sudo apt-get install -y siege

PHP_SVC_IP=$(kubectl get svc php-apache -o jsonpath='{.spec.clusterIP}')
siege -c 20 -t 2M http://$PHP_SVC_IP &
SIEGE_PID=$!

# Watch HPA scale up
kubectl get hpa php-apache -w
```

Stop the load:

```
kill $SIEGE_PID 2>/dev/null
# HPA will scale back down after ~5 minutes
kubectl get hpa php-apache -w
```

7.5.3 Clean up

```
kubectl delete hpa php-apache
kubectl delete deployment php-apache
kubectl delete svc php-apache
```

7.6 Lab 8.5 — Kustomize

Kustomize is built into `kubectl`. Create a base application and two overlays (dev and prod).

7.6.1 Create the base

```
mkdir -p ~/kustomize/base ~/kustomize/overlays/dev ~/kustomize/overlays/prod

cat <<EOF > ~/kustomize/base/deployment.yaml
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: nginx:1.25
EOF

cat <<EOF > ~/kustomize/base/kustomization.yaml
resources:
- deployment.yaml
EOF
```

7.6.2 Create the dev overlay

```
cat <<EOF > ~/kustomize/overlays/dev/kustomization.yaml
resources:
- ../../base
patches:
- patch: |-
    - op: replace
      path: /spec/replicas
      value: 1
    - op: replace
      path: /spec/template/spec/containers/0/image
      value: nginx:1.25
  target:
    kind: Deployment
    name: myapp
namePrefix: dev-
EOF
```

7.6.3 Create the prod overlay

```
cat <<EOF > ~/kustomize/overlays/prod/kustomization.yaml
resources:
- ../../base
patches:
- patch: |-
    - op: replace
      path: /spec/replicas
      value: 3
    - op: replace
      path: /spec/template/spec/containers/0/image
      value: nginx:1.26
  target:
    kind: Deployment
    name: myapp
namePrefix: prod-
EOF
```

7.6.4 Preview and apply

```
# Preview what will be applied
kubectl kustomize ~/kustomize/overlays/dev
kubectl kustomize ~/kustomize/overlays/prod

# Apply dev
kubectl apply -k ~/kustomize/overlays/dev
kubectl get deployments | grep dev-myapp

# Apply prod
kubectl apply -k ~/kustomize/overlays/prod
kubectl get deployments | grep prod-myapp
```

7.6.5 Clean up

```
kubectl delete -k ~/kustomize/overlays/dev
kubectl delete -k ~/kustomize/overlays/prod
```

8. Chapter 9: Volumes and Storage

8.1 Overview

In this lab you will:

- Use ConfigMaps and Secrets as volumes
- Create a PersistentVolume backed by NFS
- Bind a PersistentVolumeClaim and use it in a pod
- Configure ResourceQuotas for storage
- Set up dynamic provisioning with a StorageClass

Lab files are in `~/lfs458/ch09-volumes/`. Completed YAMLs for files you need to edit are in the `solutions/` subfolder.

8.2 Lab 9.1 — ConfigMaps as Volumes

Create a ConfigMap containing an nginx config:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  nginx.conf: |
    server {
      listen 80;
      root /usr/share/nginx/html;
      index index.html;
    }
EOF
```

Mount it into a pod:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nginx-configmap
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: config-vol
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: config-vol
    configMap:
      name: nginx-conf
EOF
```

```
CM_POD=$(kubectl get pod nginx-configmap -o jsonpath='{.metadata.name}')
kubectl exec $CM_POD -- cat /etc/nginx/conf.d/nginx.conf
```

Clean up:

```
kubectl delete pod nginx-configmap
kubectl delete configmap nginx-conf
```

8.3 Lab 9.2 — Using simpleshell.yaml

The `simpleshell.yaml` file defines a basic container. You will modify it in two stages.

```
cp ~/lfs458/ch09-volumes/simpleshell.yaml ~/simpleshell.yaml
cat ~/simpleshell.yaml
```

8.3.1 Stage 1 — Add envFrom

Edit `~/simpleshell.yaml` to add an `envFrom` section referencing the `nginx-conf` configmap (create a simple one first):

```
kubectl create configmap shell-config \
  --from-literal=MY_VAR=hello \
  --from-literal=ANOTHER_VAR=world
```

Add to `~/simpleshell.yaml` under `spec.containers[0]`:

```
envFrom:
- configMapRef:
    name: shell-config
```

The solution is at `~/lfs458/ch09-volumes/solutions/simpleshell-envfrom.yaml`.

Apply and verify:

```
kubectl apply -f ~/simpleshell.yaml
SS_POD=$(kubectl get pod simpleshell -o jsonpath='{.metadata.name}')
kubectl exec $SS_POD -- env | grep -E "MY_VAR|ANOTHER_VAR"
kubectl delete pod simpleshell
```

8.3.2 Stage 2 — Add a volumeMount

Edit `~/simpleshell.yaml` further to mount a volume using a `hostPath`. The solution is at `~/lfs458/ch09-volumes/solutions/simpleshell-volume.yaml`.

Apply and verify:

```
kubectl apply -f ~/simpleshell.yaml
SS_POD=$(kubectl get pod simpleshell -o jsonpath='{.metadata.name}')
kubectl exec $SS_POD -- ls /mnt/data
kubectl delete pod simpleshell
kubectl delete configmap shell-config
```

8.4 Lab 9.3 — NFS PersistentVolume

NFS Server

This lab assumes an NFS server is available. In the lab environment, the `controller` node can act as the NFS server.

8.4.1 Set up NFS server (controller only)

```
sudo apt-get install -y nfs-kernel-server
sudo mkdir -p /opt/sfw
sudo chmod 777 /opt/sfw
echo "/opt/sfw *(rw,sync,no_subtree_check,no_root_squash)" | sudo tee -a /etc/exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server
```

Verify the export:

```
showmount -e controller
```

8.4.2 Install NFS client on workers (worker1 and worker2)

```
sudo apt-get install -y nfs-common
```

8.4.3 Create the PersistentVolume

```
cp ~/lfs458/ch09-volumes/PVol.yaml ~/PVol.yaml
cat ~/PVol.yaml
```

Note the `persistentVolumeReclaimPolicy`. Edit `~/PVol.yaml` to change it to `Retain`. The solution is at `~/lfs458/ch09-volumes/solutions/PVol-retain.yaml`.

```
kubectl apply -f ~/PVol.yaml
kubectl get pv
```

8.4.4 Create a PersistentVolumeClaim

Emergency fallback: If the commands below fail, `kubectl apply -f ~/lfs458/ch09-volumes/solutions/nfs-pvc.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
  - ReadWriteMany
  resources:
    requests:
      storage: 200Mi
  storageClassName: ""
EOF

kubectl get pvc nfs-pvc
```

8.4.5 Use the PVC in a pod

Emergency fallback: If the commands below fail, `kubectl apply -f ~/lfs458/ch09-volumes/solutions/nfs-pod.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nginx-nfs
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: nfs-vol
      mountPath: /usr/share/nginx/html
  volumes:
  - name: nfs-vol
    persistentVolumeClaim:
      claimName: nfs-pvc
EOF
```

```
NFS_POD=$(kubectl get pod nginx-nfs -o jsonpath='{.metadata.name}')
kubectl describe pod $NFS_POD | grep -A5 Volumes
```

Write a test file from the controller and read it in the pod:

```
sudo bash -c 'echo "<h1>Hello from NFS</h1>" > /opt/sfw/index.html'
kubectl exec $NFS_POD -- cat /usr/share/nginx/html/index.html
```

8.4.6 Clean up

```
kubectl delete pod nginx-nfs
kubectl delete pvc nfs-pvc
kubectl delete pv pvvol-1
```

8.5 Lab 9.4 — ResourceQuota for Storage

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ResourceQuota
metadata:
  name: storage-quota
  namespace: default
spec:
  hard:
    requests.storage: "500Mi"
    persistentvolumeclaims: "3"
EOF

kubectl describe resourcequota storage-quota
```

Try to create a PVC that exceeds the quota:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: big-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 600Mi
  storageClassName: ""
EOF
```

You should see a `forbidden: exceeded quota` error.

Clean up:

```
kubectl delete resourcequota storage-quota
kubectl delete pvc big-pvc 2>/dev/null; true
```

9. Chapter 10: Services

Working directory: `~/lfs458/ch10-services/`

9.1 Exercise 10.1: Deploy A New Service

Services provide a stable network identity to a dynamic set of Pods. They are assigned labels to allow persistent access even as Pods are terminated and replaced. In this exercise we deploy nginx in a custom namespace using a `nodeSelector`, expose it as a service, and explore endpoints.

```
cd ~/lfs458/ch10-services/
```

1. Review the pre-staged `nginx-one.yaml` — a well-documented Deployment that places pods on a node labelled `system=secondOne`, in the `accounting` namespace, exposing port 8080.

```
cat ~/lfs458/ch10-services/nginx-one.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-one
  labels:
    system: secondary
  namespace: accounting
spec:
  selector:
    matchLabels:
      system: secondary
  replicas: 2
  template:
    metadata:
      labels:
        system: secondary
    spec:
      containers:
      - image: nginx:1.20.1
        imagePullPolicy: Always
        name: nginx
        ports:
        - containerPort: 8080
          protocol: TCP
      nodeSelector:
        system: secondOne
```

2. View the existing labels on the nodes in the cluster.

```
kubectl get nodes --show-labels
```

```
...
```

3. Attempt to create the Deployment. It should fail because the `accounting` namespace does not exist yet.

```
kubectl create -f nginx-one.yaml
```

```
Error from server (NotFound): error when creating
"nginx-one.yaml": namespaces "accounting" not found
```

4. Create the namespace and try again.

```
kubectl create ns accounting
```

```
namespace/accounting created
```

```
kubectl create -f nginx-one.yaml
```

```
deployment.apps/nginx-one created
```

5. View the pods in the `accounting` namespace. They show `Pending` because no node has the required label.

```
kubectl -n accounting get pods
```

```
NAME                          READY   STATUS    RESTARTS   AGE
nginx-one-74dd9d578d-fcpmv    0/1     Pending   0          4m
nginx-one-74dd9d578d-r2d67    0/1     Pending   0          4m
```

6. Describe one of the pods to confirm the scheduling failure reason.

```
kubectl -n accounting describe pod <nginx-one-pod-name>
```

```
Events:
  Type     Reason            Age    From               Message
  ----     ------            ---    ----               -------
  Warning  FailedScheduling  <unknown>  default-scheduler
  0/2 nodes are available: 2 node(s) didn't match node selector.
```

7. Label the worker1 node with the value the `nodeSelector` expects. Note the value is case sensitive.

```
kubectl label node worker1 system=secondOne
```

```
node/worker1 labeled
```

```
kubectl get nodes --show-labels
```

```
NAME     STATUS   ROLES           AGE    VERSION   LABELS
controller       Ready    control-plane   14h    v1.36.2   beta.kubernetes.io/arch=amd64,...
worker1   Ready    <none>          14h    v1.36.2   beta.kubernetes.io/arch=amd64,...,system=secondOne,...
```

8. Check the pods again — they should now be `Running`. If they still show `Pending` after a minute, delete one to trigger the scheduler to re-evaluate.

```
kubectl -n accounting get pods
```

```
NAME                          READY   STATUS    RESTARTS   AGE
nginx-one-74dd9d578d-fcpmv    1/1     Running   0          10m
nginx-one-74dd9d578d-sts5l    1/1     Running   0          3s
```

9. View Pods by the `system=secondary` label across all namespaces.

```
kubectl get pods -l system=secondary --all-namespaces
```

```
NAMESPACE    NAME                          READY   STATUS    RESTARTS   AGE
accounting   nginx-one-74dd9d578d-fcpmv    1/1     Running   0          20m
accounting   nginx-one-74dd9d578d-sts5l    1/1     Running   0          9m
```

10. Expose the deployment. Port 8080 was declared in the YAML file.

```
kubectl -n accounting expose deployment nginx-one
```

```
service/nginx-one exposed
```

11. View the newly created endpoints. Each Pod's IP is listed on port 8080.

```
kubectl -n accounting get ep nginx-one
```

Note: The `Warning: v1 Endpoints is deprecated` message is expected on Kubernetes 1.33+ and can be ignored.

```
NAME        ENDPOINTS                                AGE
nginx-one   10.244.1.5:8080,10.244.2.3:8080          47s
```

12. Test access on port 8080 (nginx is not listening there) then on port 80 (nginx default). Use the Pod IP from the endpoint output above.

```
curl 192.168.1.72:8080
```

```
curl: (7) Failed to connect to 192.168.1.72 port 8080: Connection refused
```

```
curl 192.168.1.72:80
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

13. Delete the deployment, fix the port in the YAML to `80`, and recreate it. If you get stuck, the completed file is at `~/lfs458/ch10-services/solutions/nginx-one-port80.yaml`.

```
kubectl -n accounting delete deploy nginx-one
```

```
deployment.apps "nginx-one" deleted
```

```
vim nginx-one.yaml
```

```
....
        ports:
        - containerPort: 80    #<-- Edit from 8080 to 80
          protocol: TCP
....
```

```
kubectl create -f nginx-one.yaml
```

```
deployment.apps/nginx-one created
```

9.2 Exercise 10.2: Configure a NodePort

A `NodePort` exposes the service on a static high-numbered port on every node, making it accessible from outside the cluster.

1. Expose the deployment as a NodePort service with a memorable name.

```
kubectl -n accounting expose deployment nginx-one \
  --type=NodePort --name=service-lab
```

```
service/service-lab exposed
```

2. View the service details and note the auto-generated NodePort.

```
kubectl -n accounting describe services
```

```
....
NodePort:   <unset>  32103/TCP
....
```

3. Get the cluster API server URL and confirm the CP hostname.

```
kubectl cluster-info
```

```
Kubernetes control plane is running at https://controller:6443
CoreDNS is running at https://controller:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

4. Test access to the nginx web server using the CP hostname and NodePort.

```
curl http://controller:<nodeport>
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

5. You can also access the service from outside the lab using the public IP of any node and the NodePort. Find your public IP if needed.

```
curl ifconfig.io
```

9.3 Exercise 10.3: Working with CoreDNS

CoreDNS provides DNS resolution within the cluster. Services are registered automatically using a predictable FQDN pattern: `<service>.<namespace>.svc.cluster.local`.

1. Review the pre-staged `nettool.yaml` — a long-running Ubuntu Pod for network diagnostics.

```
cat ~/lfs458/ch10-services/nettool.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu
spec:
  containers:
  - name: ubuntu
    image: ubuntu:latest
    command: [ "sleep" ]
    args: [ "infinity" ]
```

2. Create the Pod and exec into it.

```
kubectl create -f nettool.yaml
```

```
pod/ubuntu created
```

```
kubectl exec -it ubuntu -- /bin/bash
```

The following sub-steps **(a)–(h)** run **inside the ubuntu container**.

(a) Install network tools.

```
apt-get update ; apt-get install curl dnsutils -y
```

(b) Run `dig` with no arguments to see the DNS server responding.

```
dig
```

```
; <<>> DiG 9.16.1-Ubuntu <<>>
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3394
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; Query time: 4 msec
...
```

(c) Check `/etc/resolv.conf` — the first search domain is `default.svc.cluster.local` since the Pod is in the `default` namespace.

```
cat /etc/resolv.conf
```

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

(d) Do a reverse lookup on the CoreDNS IP. Note the domain uses `.kube-system.svc.cluster.local.` — reflecting the pod's namespace.

```
dig @10.96.0.10 -x 10.96.0.10
```

```
;; ANSWER SECTION:
10.0.96.10.in-addr.arpa.   30  IN  PTR  kube-dns.kube-system.svc.cluster.local.
```

(e) Use the full FQDN of the `service-lab` service to access nginx. This works because the service is in the `accounting` namespace.

```
curl service-lab.accounting.svc.cluster.local.
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

(f) Try using just the short service name. It fails because `nettool` is in the `default` namespace and `service-lab` is in `accounting`.

```
curl service-lab
```

```
curl: (6) Could not resolve host: service-lab
```

(g) Add the namespace to the short name and try again.

```
curl service-lab.accounting
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

(h) Exit the container.

```
exit
```

3. Examine the `kube-dns` service in `kube-system`. Note the selector that routes DNS traffic.

```
kubectl -n kube-system get svc kube-dns -o yaml
```

```
...
  labels:
    k8s-app: kube-dns
    kubernetes.io/cluster-service: "true"
    kubernetes.io/name: CoreDNS
...
  selector:
    k8s-app: kube-dns
  sessionAffinity: None
  type: ClusterIP
...
```

4. Find all pods with the `k8s-app` label across all namespaces — these are the infrastructure pods served by this selector, including `coredns`.

```
kubectl get pod -l k8s-app --all-namespaces
```

```
NAMESPACE     NAME                              READY   STATUS    RESTARTS   AGE
kube-system   coredns-5d78c9869d-44qvq          1/1     Running   0          31m
kube-system   coredns-5d78c9869d-j6tqx          1/1     Running   0          31m
kube-system   kube-proxy-lpsmq                  1/1     Running   0          35m
kube-system   kube-proxy-pvl8w                  1/1     Running   0          34m
```

5. Inspect a CoreDNS pod. Note the configuration is loaded from a ConfigMap mounted at `/etc/coredns/Corefile`.

```
COREDNS_POD=$(kubectl -n kube-system get pod -l k8s-app=kube-dns \
  -o jsonpath='{.items[0].metadata.name}')
kubectl -n kube-system get pod $COREDNS_POD -o yaml
```

```
...
spec:
  containers:
  - args:
    - -conf
    - /etc/coredns/Corefile
    image: k8s.gcr.io/coredns:1.7.0
...
    volumeMounts:
    - mountPath: /etc/coredns
      name: config-volume
```

```
      readOnly: true
...
  volumes:
  - configMap:
      items:
      - key: Corefile
        path: Corefile
      name: coredns
    name: config-volume
...
```

6. View the ConfigMaps in `kube-system`. The `coredns` ConfigMap holds the Corefile.

```
kubectl -n kube-system get configmaps
```

```
NAME                                 DATA   AGE
coredns                              1      43h
extension-apiserver-authentication   6      43h
kube-proxy                           2      43h
kubeadm-config                       2      43h
kubelet-config                       1      43h
```

7. View the CoreDNS Corefile configuration. Note `cluster.local` is the default domain and `forward` sends unknown queries to `/etc/resolv.conf`.

```
kubectl -n kube-system get configmaps coredns -o yaml
```

```
apiVersion: v1
data:
  Corefile: |
    .:53 {
        errors
        health {
          lameduck 5s
        }
        ready
        kubernetes cluster.local in-addr.arpa ip6.arpa {
          pods insecure
          fallthrough in-addr.arpa ip6.arpa
          ttl 30
        }
        prometheus :9153
        forward . /etc/resolv.conf {
          max_concurrent 1000
        }
        cache 30
        loop
        reload
        loadbalance
    }
kind: ConfigMap
...
```

8. Back up the CoreDNS ConfigMap before editing.

```
kubectl -n kube-system get configmaps coredns -o yaml > coredns-backup.yaml
```

9. Edit the CoreDNS ConfigMap to add a `rewrite` rule so that `*.test.io` domains resolve to `*.default.svc.cluster.local`. Add the rewrite line as the **first** entry inside the `.:53` block.

```
kubectl -n kube-system edit configmaps coredns
```

```
....
    .:53 {
        rewrite name regex (.*)\.test\.io {1}.default.svc.cluster.local  #<-- Add this line
        errors
        health {
          lameduck 5s
        }
....
```

10. Delete the CoreDNS pods to force them to re-read the updated ConfigMap.

```
kubectl -n kube-system delete pod <coredns-pod-1> <coredns-pod-2>
```

```
pod "coredns-f9fd979d6-s4j98" deleted
pod "coredns-f9fd979d6-xlpzf" deleted
```

11. Create an nginx deployment and expose it as a ClusterIP service to test the rewrite rule.

```
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --type=ClusterIP --port=80
kubectl get svc
```

```
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP   3d15h
nginx        ClusterIP   10.104.248.141   <none>        80/TCP    7s
```

12. Exec into the ubuntu container and test the rewrite rule.

```
kubectl exec -it ubuntu -- /bin/bash
```

(a) Do a reverse lookup to get the FQDN of the nginx service IP.

```
dig -x 10.104.248.141
```

```
;; ANSWER SECTION:
141.248.104.10.in-addr.arpa.  30  IN  PTR  nginx.default.svc.cluster.local.
```

(b) Do a forward lookup using the full FQDN.

```
dig nginx.default.svc.cluster.local.
```

```
;; ANSWER SECTION:
nginx.default.svc.cluster.local.  30  IN  A  10.104.248.141
```

(c) Test the `test.io` rewrite rule — `nginx.test.io` should resolve to the same IP as `nginx.default.svc.cluster.local`.

```
dig nginx.test.io
```

```
;; ANSWER SECTION:
nginx.default.svc.cluster.local.  30  IN  A  10.104.248.141
```

(d) Exit the container.

```
exit
```

13. Edit the CoreDNS configmap again to upgrade the rewrite rule to also return the `test.io` name in the answer (using `rewrite stop` with an `answer` clause).

```
kubectl -n kube-system edit configmaps coredns
```

```
....
    .:53 {
        rewrite stop {                                              #<-- Edit these 3 lines
          name regex (.*)\.test\.io {1}.default.svc.cluster.local
          answer name (.*)\.default\.svc\.cluster\.local {1}.test.io
          }
        errors
        health {
....
```

14. Delete the CoreDNS pods again to pick up the new config.

```
kubectl -n kube-system delete pod <coredns-pod-1> <coredns-pod-2>
```

15. Exec into ubuntu and test again. This time the answer section should return the `test.io` FQDN rather than the internal one.

```
kubectl exec -it ubuntu -- /bin/bash
```

```
dig nginx.test.io
```

```
;; ANSWER SECTION:
nginx.test.io.   30  IN  A  10.104.248.141
```

```
exit
```

16. Delete the ubuntu nettool container to free up resources.

```
kubectl delete -f nettool.yaml
```

9.4 Exercise 10.4: Use Labels to Manage Resources

Labels provide a powerful way to select and manage groups of resources across namespaces.

1. Delete all Pods with the `system=secondary` label across all namespaces. The Deployment controller will immediately create replacement Pods.

```
kubectl delete pods -l system=secondary \
  --all-namespaces
```

```
pod "nginx-one-74dd9d578d-fcpmv" deleted
pod "nginx-one-74dd9d578d-sts5l" deleted
```

2. Check the `accounting` namespace — new Pods are already running.

```
kubectl -n accounting get pods
```

```
NAME                         READY   STATUS    RESTARTS   AGE
nginx-one-74dd9d578d-ddt5r   1/1     Running   0          1m
nginx-one-74dd9d578d-hfzml   1/1     Running   0          1m
```

3. View the deployment label to confirm it still carries the `system=secondary` label.

```
kubectl -n accounting get deploy --show-labels
```

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE   LABELS
nginx-one   2/2     2            2           10m   system=secondary
```

4. Delete the deployment using its label.

```
kubectl -n accounting delete deploy -l system=secondary
```

```
deployment.apps "nginx-one" deleted
```

5. Remove the `system` label from the worker1 node. The syntax is the key name followed immediately by a minus sign.

```
kubectl label node worker1 system-
```

```
node/worker1 unlabeled
```

10. Chapter 11: Ingress and Gateway API

10.1 Overview

In this lab you will:

- Install the NGINX Ingress Controller as a DaemonSet
- Create Ingress rules to route traffic to multiple backends
- Explore the Gateway API as the next-generation Ingress alternative

---

10.2 Lab 11.1 — Install NGINX Ingress Controller

Deploy NGINX Ingress as a DaemonSet so it runs on every node:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.1/deploy/static/provider/baremetal/deploy.yaml
```

Wait for the controller to be ready:

```
kubectl rollout status deployment ingress-nginx-controller -n ingress-nginx --timeout=120s
kubectl get pods -n ingress-nginx
```

Get the NodePort assigned to the ingress controller:

```
HTTP_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
HTTPS_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
  -o jsonpath='{.spec.ports[?(@.name=="https")].nodePort}')

echo "HTTP  NodePort: $HTTP_PORT"
echo "HTTPS NodePort: $HTTPS_PORT"
```

10.3 Lab 11.2 — Create Ingress Rules

10.3.1 Deploy two backend applications

```
kubectl create deployment app1 --image=nginx --replicas=2
kubectl expose deployment app1 --port=80

kubectl create deployment app2 --image=httpd --replicas=2
kubectl expose deployment app2 --port=80
```

10.3.2 Create an Ingress resource

Emergency fallback: If the heredoc below fails, `kubectl apply -f ~/lfs458/ch11-ingress/solutions/apps-ingress.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: apps-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app1.vega.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1
```

```
            port:
              number: 80
  - host: app2.vega.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app2
            port:
              number: 80
EOF
```

10.3.3 Test the Ingress

Add entries to `/etc/hosts` on the controller (or wherever you're running curl):

```
CONTROLLER_IP=$(kubectl get node controller -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}')
echo "$CONTROLLER_IP app1.vega.local app2.vega.local" | sudo tee -a /etc/hosts
```

Test:

```
HTTP_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')

curl http://app1.vega.local:$HTTP_PORT
curl http://app2.vega.local:$HTTP_PORT
```

`app1` should return the nginx welcome page; `app2` should return the Apache (httpd) page.

10.3.4 Clean up

```
kubectl delete ingress apps-ingress
kubectl delete deployment app1 app2
kubectl delete svc app1 app2
```

10.4 Lab 11.3 — Gateway API

The Gateway API is the successor to Ingress. It separates infrastructure configuration (Gateway) from routing rules (HTTPRoute).

10.4.1 Install Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.0/standard-install.yaml
kubectl get crd | grep gateway
```

10.4.2 Install NGINX Gateway Fabric

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.5.1/deploy/crds.yaml
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.5.1/deploy/nodeport/deploy.yaml

kubectl rollout status deployment nginx-gateway -n nginx-gateway --timeout=120s
```

Get the Gateway ports:

```
GW_HTTP=$(kubectl get svc nginx-gateway -n nginx-gateway \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
echo "Gateway HTTP NodePort: $GW_HTTP"
```

10.4.3 Create a GatewayClass and Gateway

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
```

```
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
  namespace: default
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP
EOF
```

10.4.4 Create a backend and HTTPRoute

```
kubectl create deployment gw-app --image=nginx
kubectl expose deployment gw-app --port=80

cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: gw-app-route
spec:
  parentRefs:
  - name: main-gateway
  hostnames:
  - "gw-app.vega.local"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: gw-app
      port: 80
EOF
```

Emergency fallbacks: `kubectl apply -f ~/lfs458/ch11-ingress/solutions/main-gateway.yaml` and `kubectl apply -f ~/lfs458/ch11-ingress/solutions/gw-app-route.yaml`

Test:

```
echo "$CONTROLLER_IP gw-app.vega.local" | sudo tee -a /etc/hosts
curl http://gw-app.vega.local:$GW_HTTP
```

10.4.5 Clean up

```
kubectl delete httproute gw-app-route
kubectl delete gateway main-gateway
kubectl delete gatewayclass nginx
kubectl delete deployment gw-app
kubectl delete svc gw-app
```

11. Chapter 12: Scheduling

11.1 Overview

In this lab you will:

- Use `nodeSelector` to pin pods to specific nodes
- Work with node labels
- Apply Taints and Tolerations (`PreferNoSchedule` , `NoSchedule` , `NoExecute`)
- Understand how the scheduler makes placement decisions

11.2 Lab 12.1 — nodeSelector

Label a node:

```
kubectl label node worker1 disktype=ssd
kubectl get nodes --show-labels | grep disktype
```

Create a pod that requires `disktype=ssd` :

Emergency fallback: `kubectl apply -f ~/lfs458/ch12-scheduling/solutions/ssd-pod.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: nginx
    image: nginx
EOF
```

Verify it landed on `worker1` :

```
kubectl get pod ssd-pod -o wide
```

Delete and remove the label:

```
kubectl delete pod ssd-pod
kubectl label node worker1 disktype-
```

11.3 Lab 12.2 — Node Affinity

Node Affinity is a more expressive replacement for `nodeSelector` .

Label nodes:

```
kubectl label node worker1 zone=west
kubectl label node worker2 zone=east
```

Create a pod with `preferredDuringSchedulingIgnoredDuringExecution` (soft requirement):

Emergency fallback: `kubectl apply -f ~/lfs458/ch12-scheduling/solutions/affinity-pod.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
```

```
metadata:
  name: affinity-pod
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: zone
            operator: In
            values:
            - west
  containers:
  - name: nginx
    image: nginx
EOF
```

```
kubectl get pod affinity-pod -o wide
```

Clean up:

```
kubectl delete pod affinity-pod
kubectl label node worker1 zone-
kubectl label node worker2 zone-
```

11.4 Lab 12.3 — Taints and Tolerations

11.4.1 PreferNoSchedule

Apply a `PreferNoSchedule` taint to `worker2` — the scheduler will try to avoid it but will use it if necessary:

```
kubectl taint node worker2 gpu=true:PreferNoSchedule
kubectl describe node worker2 | grep -A3 Taints
```

Deploy many replicas and observe where they land:

```
kubectl create deployment no-pref --image=nginx --replicas=6
kubectl get pods -l app=no-pref -o wide
```

Most pods should land on `controller` and `worker1`; some may land on `worker2`.

Remove the taint:

```
kubectl taint node worker2 gpu=true:PreferNoSchedule-
kubectl delete deployment no-pref
```

11.4.2 NoSchedule

`NoSchedule` is a hard rule — no new pods will be scheduled on the tainted node unless they tolerate the taint.

```
kubectl taint node worker2 gpu=true:NoSchedule
```

Deploy without a toleration — pods should NOT land on `worker2`:

```
kubectl create deployment no-sched --image=nginx --replicas=4
kubectl get pods -l app=no-sched -o wide
```

Now deploy with a toleration:

Emergency fallback: `kubectl apply -f ~/lfs458/ch12-scheduling/solutions/tolerates-gpu.yaml`

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tolerates-gpu
spec:
  replicas: 4
  selector:
    matchLabels:
```

```
      app: tolerates-gpu
  template:
    metadata:
      labels:
        app: tolerates-gpu
    spec:
      tolerations:
      - key: gpu
        operator: Equal
        value: "true"
        effect: NoSchedule
      containers:
      - name: nginx
        image: nginx
EOF
```

```
kubectl get pods -l app=tolerates-gpu -o wide
```

These pods can land on `worker2`.

Remove the taint and clean up:

```
kubectl taint node worker2 gpu=true:NoSchedule-
kubectl delete deployment no-sched tolerates-gpu
```

11.4.3 NoExecute

`NoExecute` evicts already-running pods that lack the toleration.

```
kubectl create deployment evict-test --image=nginx --replicas=4
kubectl get pods -l app=evict-test -o wide
```

Note some pods on `worker2`. Now taint it with `NoExecute`:

```
kubectl taint node worker2 maintenance=true:NoExecute
kubectl get pods -l app=evict-test -o wide -w
```

Pods on `worker2` will be evicted immediately and rescheduled on the other nodes.

Remove the taint and clean up:

```
kubectl taint node worker2 maintenance=true:NoExecute-
kubectl delete deployment evict-test
```

12. Chapter 13: Logging and Troubleshooting

Working directory: `~/lfs458/ch13-troubleshoot/`

12.1 Exercise 13.1: Review Log File Locations

Kubernetes distributes logs across nodes and containers. In this exercise we identify the key log locations and understand how to access them.

```
cd ~/lfs458/ch13-troubleshoot/
```

1. View the **kubelet** service logs using `journalctl`. The kubelet is the primary node agent and its logs are essential for diagnosing pod scheduling and startup issues.

```
journalctl -u kubelet | less
```

```
...
```

2. Major Kubernetes processes run inside containers. Use `find` to locate the **kube-apiserver** log on disk.

```
sudo find / -name "*apiserver*log"
```

```
/var/log/containers/kube-apiserver-controller_kube-system_kube-apiserver-423
d25701998f68b503e64d41dd786e657fc09504f13278044934d79a4019e3c.log
```

3. View the apiserver log file directly.

```
sudo less /var/log/containers/kube-apiserver-controller_kube-system_kube-apiserver-<Tab>
```

```
...
```

4. Search for and review log files for other cluster agents: `coredns`, `kube-proxy`, and others.

```
sudo find / -name "*coredns*log" 2>/dev/null
sudo find / -name "*kube-proxy*log" 2>/dev/null
```

5. On **systemd**-based clusters (all modern kubeadm deployments), component logs are in `journalctl`. On older non-systemd systems the log files below would be used instead — know these paths for reference:

Control Plane node log files (non-systemd only): - `/var/log/kube-apiserver.log` — API server - `/var/log/kube-scheduler.log` — scheduling decisions - `/var/log/kube-controller-manager.log` — replication controllers

Worker node log files (non-systemd only): - `/var/log/kubelet.log` — container runtime on the node - `/var/log/kube-proxy.log` — service load balancing

Container log files (all systems): - `/var/log/containers/` — individual container log files (symlinked) - `/var/log/pods/` — pod-level log directories

6. Review the Kubernetes documentation for more troubleshooting information: - https://kubernetes.io/docs/tasks/debug-application-cluster/troubleshooting/ - https://kubernetes.io/docs/tasks/debug-application-cluster/debug-application - https://kubernetes.io/docs/tasks/debug-application-cluster/debug-cluster

12.2 Exercise 13.2: Viewing Logs Output

Container standard output is accessible via `kubectl logs`. If a container writes no stdout there will be no log output. Logs are also lost when the container is destroyed unless an external logging agent is used.

1. View all pods running in the cluster across all namespaces.

```
kubectl get po --all-namespaces
```

```
NAMESPACE     NAME                                       READY   STATUS    RESTARTS   AGE
calico-system   calico-kube-controllers-788c7d7585-jgn6s   1/1     Running   0          13m
calico-system   calico-node-5tv9d                          1/1     Running   0          6d1h
....
kube-system   etcd-controller                              1/1     Running   2          44h
kube-system   kube-apiserver-controller                    1/1     Running   2          44h
kube-system   kube-controller-manager-controller           1/1     Running   2          44h
kube-system   kube-scheduler-controller                    1/1     Running   2          44h
....
```

2. View the logs of infrastructure pods. Use Tab completion — first type the namespace, then start typing the pod name and Tab will complete it.

```
kubectl -n kube-system logs <Tab><Tab>
```

```
calico-kube-controllers-788c7d7585-jgn6s
calico-node-5tv9d
coredns-5644d7b6d9-k7kts
coredns-5644d7b6d9-rnr2v
etcd-controller
kube-apiserver-controller
kube-controller-manager-controller
kube-proxy-qhc4f
kube-proxy-s56hl
kube-scheduler-f-controller
...
```

```
kubectl -n kube-system logs \
  kube-apiserver-controller
```

```
Flag --insecure-port has been deprecated, This flag will be removed in a future version.
I1119 02:31:14.933023   1 server.go:623] external host was not specified, using 10.128.0.3
I1119 02:31:14.933356   1 server.go:149] Version: v1.36.2
I1119 02:31:15.595131   1 plugins.go:158] Loaded 11 mutating admission controller(s)
...
```

3. View the logs of other pods in your cluster. The `--previous` flag shows logs from a previously crashed container, which is essential for post-mortem debugging.

```
kubectl -n kube-system logs kube-scheduler-f-controller
kubectl -n kube-system logs etcd-controller
```

12.3 Exercise 13.3: Adding Tools for Monitoring and Metrics

The **Metrics Server** exposes a standard API that `kubectl top` and the HPA controller use. It was installed in Chapter 8 (Lab 8.3), so here we only verify it and then query it.

12.3.1 Verify Metrics Server

Note: The metrics-server was already installed in Lab 8.3 via Helm with the `--kubelet-insecure-tls` flag. Verify it is still running before continuing — do NOT reinstall it.

1. Confirm the metrics-server pod is running.

```
kubectl -n kube-system get pods | grep metrics-server
```

```
metrics-server-fc6d4999b-b9rjj   1/1   Running   0   2d
```

If the pod shows `0/1` or is missing, go back to Lab 8.3 and reinstall via Helm.

2. Test that metrics are working. It can take up to a minute for metrics to populate.

```
sleep 120 ; kubectl top pod --all-namespaces
```

```
NAMESPACE     NAME                                       CPU(cores)   MEMORY(bytes)
calico-system   calico-kube-controllers-7b9dcdcc5-qg6zd   2m           6Mi
calico-system   calico-node-dr279                          23m          22Mi
kube-system   coredns-5644d7b6d9-k7kts                  2m           6Mi
...
```

```
kubectl top nodes
```

```
NAME     CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
controller       228m         11%    2357Mi          31%
worker1   76m          3%     1385Mi          18%
```

3. You can also query the metrics API directly using TLS certificates from your kubeconfig. Generate the PEM files first, then run the curl.

```
export client=$(grep client-cert $HOME/.kube/config | cut -d" " -f 6)
export key=$(grep client-key-data $HOME/.kube/config | cut -d" " -f 6)
export auth=$(grep certificate-authority-data $HOME/.kube/config | cut -d" " -f 6)
echo $client | base64 -d - > ./client.pem
echo $key | base64 -d - > ./client-key.pem
echo $auth | base64 -d - > ./ca.pem
```

```
curl --cert ./client.pem \
  --key ./client-key.pem --cacert ./ca.pem \
  https://controller:6443/apis/metrics.k8s.io/v1beta1/nodes
```

```
{
  "kind": "NodeMetricsList",
  "apiVersion": "metrics.k8s.io/v1beta1",
  "metadata": {
    "selfLink": "/apis/metrics.k8s.io/v1beta1/nodes"
  },
  "items": [
    {
      "metadata": {
        "name": "u16-1-13-1-2f8c",
        "timestamp": "2024-08-10T20:26:18Z",
        "window": "30s",
      },
      "usage": {
        "cpu": "215675721n",
        "memory": "2414744Ki"
      }
    },
...
```

12.3.2 Configure the Kubernetes Dashboard

The Kubernetes Dashboard provides a web-based GUI for cluster management.

1. Deploy the dashboard using the pre-staged YAML.

```
kubectl create -f dashboard.yaml
```

2. Grant the dashboard service account cluster-admin rights so it can view all resources.

```
kubectl get sa -n kubernetes-dashboard
```

```
NAME                   SECRETS   AGE
default                0         4m25s
kubernetes-dashboard   0         4m25s
```

```
kubectl create clusterrolebinding dashaccess \
  --clusterrole=cluster-admin \
  --serviceaccount=kubernetes-dashboard:kubernetes-dashboard
```

```
clusterrolebinding.rbac.authorization.k8s.io/dashaccess created
```

3. Find the public IP of the CP node and find the dashboard's NodePort, then open a browser to the HTTPS URL. You will get a security exception — click **Advanced** then **Add Exception** (or equivalent in your browser).

```
curl ifconfig.io
kubectl -n kubernetes-dashboard get svc
```

Browse to: `https://<your-public-ip>:<nodeport>/`

4. Select the **Token** authentication method. Generate a token for the dashboard service account and paste it into the login page.

```
kubectl create token kubernetes-dashboard \
  -n kubernetes-dashboard
```

```
eyJlxvezoLAilithbGciOiJSUzI1NiIsImtpZCI6IiJ9.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2...
...
```

5. After logging in, explore the dashboard. Switch namespace to `kube-system` to see infrastructure pods and their CPU/memory usage graphs. Try scaling a deployment up and down and observe the dashboard respond in real time.

13. Chapter 14: Custom Resource Definition

Working directory: `~/lfs458/ch14-crd/`

13.1 Exercise 14.1: Create a Custom Resource Definition

A **Custom Resource Definition (CRD)** extends the Kubernetes API with new object types without writing a full API server. Once a CRD is registered, you can create, list, describe and delete instances of the new resource using standard `kubectl` commands.

```
cd ~/lfs458/ch14-crd/
```

1. View the existing CRDs in the cluster. You will see CRDs created by Calico from previous labs.

```
kubectl get crd --all-namespaces
```

```
NAME                                        CREATED AT
authorizationpolicies.policy.linkerd.io     2024-08-28T11:30:34Z
bgpconfigurations.crd.projectcalico.org     2024-08-28T08:58:54Z
bgppeers.crd.projectcalico.org              2024-08-28T08:58:57Z
...
```

2. Examine one of the existing CRDs to understand their structure. Copy the Calico custom resources YAML from Chapter 3 and inspect it, then describe the CRD to see the full spec.

```
cp ~/lfs458/ch03-install/calico-custom-resources.yaml .
less calico-custom-resources.yaml
kubectl describe crd installations.operator.tigera.io
```

```
Name:         installations.operator.tigera.io
Namespace:
Labels:       <none>
API Version:  apiextensions.k8s.io/v1
Kind:         CustomResourceDefinition
...
```

3. Review the pre-staged `crd.yaml` file. It defines a new `CronTab` resource in the `stable.example.com` group with OpenAPI v3 schema validation.

```
cat ~/lfs458/ch14-crd/crd.yaml
```

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # name must match the spec fields below: <plural>.<group>
  name: crontabs.stable.example.com
spec:
  # group name for REST API: /apis/<group>/<version>
  group: stable.example.com
  versions:
    - name: v1
      # Each version can be enabled/disabled by the Served flag.
      served: true
      # One and only one version must be marked as the storage version.
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                cronSpec:
                  type: string
                image:
                  type: string
                replicas:
                  type: integer
  # either Namespaced or Cluster
  scope: Namespaced
  names:
```

```
    # plural name used in the URL: /apis/<group>/<version>/<plural>
    plural: crontabs
    # singular name used as an alias on the CLI and for display
    singular: crontab
    # kind is normally the CamelCased singular type
    kind: CronTab
    # shortNames allow shorter string to match your resource on the CLI
    shortNames:
    - ct
```

4. Register the new CRD with the cluster.

```
kubectl create -f crd.yaml
```

```
customresourcedefinition.apiextensions.k8s.io/crontabs.stable.example.com created
```

5. List all CRDs and verify the new one appears. Then describe it to see the full details.

```
kubectl get crd
```

```
NAME                                  CREATED AT
...
crontabs.stable.example.com           2024-08-13T03:18:07Z
...
```

```
kubectl describe crd crontab<Tab>
```

```
Name:         crontabs.stable.example.com
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  apiextensions.k8s.io/v1
Kind:         CustomResourceDefinition
...
```

6. Review the pre-staged `new-crontab.yaml` — an instance of the new `CronTab` resource type. The `apiVersion` uses the group and version from the CRD, and `kind` matches the CamelCased kind.

```
cat ~/lfs458/ch14-crd/new-crontab.yaml
```

```
apiVersion: "stable.example.com/v1"
     # This is from the group and version of new CRD
kind: CronTab
     # The kind from the new CRD
metadata:
  name: new-cron-object
spec:
  cronSpec: "*/5 * * * *"
  image: some-cron-image
     #Does not exist
```

7. Create the new CronTab object and verify it can be accessed using both the full kind and the shortname `ct`.

```
kubectl create -f new-crontab.yaml
```

```
crontab.stable.example.com/new-cron-object created
```

```
kubectl get CronTab
```

```
NAME              AGE
new-cron-object   22s
```

```
kubectl get ct
```

```
NAME              AGE
new-cron-object   29s
```

```
kubectl describe ct
```

```
Name:         new-cron-object
Namespace:    default
Labels:       <none>
Annotations:  <none>
API Version:  stable.example.com/v1
Kind:         CronTab
```

```
...
Spec:
  Cron Spec:  */5 * * * *
  Image:      some-cron-image
Events:       <none>
```

8. Delete the CRD. This automatically removes all instances of the resource as well.

```
kubectl delete -f crd.yaml
```

```
customresourcedefinition.apiextensions.k8s.io "crontabs.stable.example.com" deleted
```

14. Chapter 15: Security

14.1 Overview

In this lab you will:

- Create a new user with TLS certificates (DevDan)
- Configure RBAC: Roles, ClusterRoles, RoleBindings
- Restrict a namespace with a NetworkPolicy
- Explore Admission Controllers

Lab files are in `~/lfs458/ch15-security/`. The `solutions/` subfolder contains completed manifests.

14.2 Lab 15.1 — Create a User with TLS Certificates

Kubernetes doesn't manage user accounts directly — users are identified by the Common Name (CN) in their TLS certificate, signed by the cluster CA.

14.2.1 Generate a key and CSR for DevDan

```
mkdir -p ~/devdan
openssl genrsa -out ~/devdan/devdan.key 2048

openssl req -new \
  -key ~/devdan/devdan.key \
  -out ~/devdan/devdan.csr \
  -subj "/CN=devdan/O=developers"
```

14.2.2 Submit a CertificateSigningRequest to Kubernetes

```
CSR_BASE64=$(cat ~/devdan/devdan.csr | base64 | tr -d '\n')

cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: devdan
spec:
  request: $CSR_BASE64
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages:
  - client auth
EOF
```

Approve the CSR:

```
kubectl certificate approve devdan
kubectl get csr devdan
```

Retrieve the signed certificate:

```
kubectl get csr devdan -o jsonpath='{.status.certificate}' | base64 -d > ~/devdan/devdan.crt
```

14.2.3 Add DevDan to kubeconfig

```
kubectl config set-credentials devdan \
  --client-certificate=$HOME/devdan/devdan.crt \
  --client-key=$HOME/devdan/devdan.key

kubectl config set-context devdan-context \
  --cluster=$(kubectl config view -o jsonpath='{.clusters[0].name}') \
  --user=devdan
```

```
kubectl config get-contexts
```

Test that DevDan currently has no permissions:

```
kubectl get pods --context=devdan-context
```

You should see a `Forbidden` error.

14.3 Lab 15.2 — RBAC: Role and RoleBinding

14.3.1 Create a namespace for DevDan

```
kubectl create namespace prod
```

14.3.2 Create a Role in the prod namespace

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: prod
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
EOF
```

14.3.3 Bind the Role to DevDan

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devdan-pod-reader
  namespace: prod
subjects:
- kind: User
  name: devdan
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
EOF
```

14.3.4 Test DevDan's access

```
# Update the devdan context to use the prod namespace
kubectl config set-context devdan-context \
  --namespace=prod \
  --cluster=$(kubectl config view -o jsonpath='{.clusters[0].name}') \
  --user=devdan

# DevDan can list pods in prod
kubectl get pods --context=devdan-context -n prod

# DevDan cannot create pods
kubectl run test --image=nginx --context=devdan-context -n prod
```

The last command should return `Forbidden`.

14.3.5 Grant more permissions (edit the Role)

Edit the role to also allow creating pods. The solution is at `~/lfs458/ch15-security/solutions/role-prod-create.yaml`.

```
kubectl edit role pod-reader -n prod
# Add "create", "delete" to the verbs list
```

14.4 Lab 15.3 — ClusterRole and ClusterRoleBinding

Create a read-only ClusterRole and bind it to DevDan cluster-wide:

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-viewer
rules:
- apiGroups: [""]
  resources: ["nodes", "namespaces", "persistentvolumes"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: devdan-cluster-viewer
subjects:
- kind: User
  name: devdan
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-viewer
  apiGroup: rbac.authorization.k8s.io
EOF
```

Test:

```
kubectl get nodes --context=devdan-context
kubectl get namespaces --context=devdan-context
kubectl get secrets --context=devdan-context -n default  # Should be Forbidden
```

14.4.1 Check permissions with auth can-i

```
kubectl auth can-i list pods --as=devdan -n prod
kubectl auth can-i create secrets --as=devdan -n prod
kubectl auth can-i get nodes --as=devdan
```

14.5 Lab 15.4 — NetworkPolicy

By default, pods can communicate freely across namespaces. NetworkPolicies restrict this.

Create two namespaces and deploy pods:

```
kubectl create namespace frontend
kubectl create namespace backend

kubectl run frontend-pod --image=nginx -n frontend --labels=app=frontend
kubectl run backend-pod --image=nginx -n backend --labels=app=backend
```

Allow only the `frontend` namespace to reach `backend` on port 80:

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: frontend
    ports:
    - protocol: TCP
```

```
      port: 80
EOF
```

Test connectivity:

```
BACKEND_IP=$(kubectl get pod backend-pod -n backend -o jsonpath='{.status.podIP}')
echo "Backend pod IP: $BACKEND_IP"

# From frontend — should work
kubectl exec -n frontend frontend-pod -- curl -s --max-time 5 http://$BACKEND_IP | head -5

# From default namespace — should be blocked
kubectl run testpod --image=busybox --rm -i --restart=Never -- \
  wget -O- --timeout=5 http://$BACKEND_IP
```

14.5.1 Clean up

```
kubectl delete networkpolicy allow-frontend -n backend
kubectl delete pod frontend-pod -n frontend
kubectl delete pod backend-pod -n backend
kubectl delete namespace frontend backend
```

14.6 Lab 15.5 — Clean Up

```
kubectl delete rolebinding devdan-pod-reader -n prod
kubectl delete role pod-reader -n prod
kubectl delete clusterrolebinding devdan-cluster-viewer
kubectl delete clusterrole cluster-viewer
kubectl delete namespace prod
kubectl delete csr devdan
kubectl config delete-context devdan-context
kubectl config delete-user devdan
```

15. Chapter 16: High Availability

15.1 Overview

In this lab you will:

- Set up HAProxy as a load balancer in front of multiple control plane nodes
- Join a second control plane node to the cluster
- Test etcd leader election
- Simulate a control plane failure and verify workloads continue running

> **Lab topology**
>
> This chapter assumes you have access to an additional VM for a second control plane node. If not, you can still follow the HAProxy and etcd sections on the existing 3-node cluster.

15.2 Lab 16.1 — HAProxy Load Balancer

HAProxy will distribute API server traffic across all control plane nodes on port 6443.

Note: HAProxy must run on a **dedicated load balancer node** — it cannot run on the controller itself. Both HAProxy and `kube-apiserver` bind to port 6443, so they cannot coexist on the same host. This lab requires a 4th VM. In a standard 3-node training environment this lab is demonstrated by the instructor rather than executed by students.

15.2.1 Install HAProxy (dedicated LB node required)

```
sudo apt-get install -y haproxy
```

15.2.2 Configure HAProxy

Replace `/etc/haproxy/haproxy.cfg` with:

```
sudo tee /etc/haproxy/haproxy.cfg > /dev/null <<'EOF'
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon

defaults
    log     global
    mode    tcp
    option  tcplog
    option  dontlognull
    timeout connect 5s
    timeout client  50s
    timeout server  50s

frontend kubernetes-api
    bind *:6443
    mode tcp
    default_backend kubernetes-masters

backend kubernetes-masters
    mode tcp
    balance roundrobin
    option tcp-check
    server controller controller:6443 check
    # Add additional control plane nodes here:
    # server controller2 controller2:6443 check
EOF

sudo systemctl restart haproxy
```

```
sudo systemctl enable haproxy
sudo systemctl status haproxy
```

Verify HAProxy is listening:

```
ss -tlnp | grep 6443
```

15.3 Lab 16.2 — Add a Second Control Plane Node

On the **controller**, generate a new join command with the `--control-plane` flag:

```
# Generate a new certificate key
CERT_KEY=$(kubeadm certs certificate-key)
echo "Certificate key: $CERT_KEY"

# Upload certs
sudo kubeadm init phase upload-certs --upload-certs --certificate-key $CERT_KEY

# Generate the join command
kubeadm token create --print-join-command --certificate-key $CERT_KEY
```

On the **second control plane node** (e.g., `controller2`), run the join command with `--control-plane`:

```
sudo kubeadm join <load-balancer-ip>:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Set up kubectl on the new node:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Back on the **original controller**, verify:

```
kubectl get nodes
kubectl get pods -n kube-system | grep etcd
```

You should now see two etcd pods and two `controller` nodes.

15.4 Lab 16.3 — etcd Leader Election

Inspect the etcd cluster members:

```
ETCD_POD=$(kubectl get pod -n kube-system -l component=etcd \
  -o jsonpath='{.items[0].metadata.name}')

kubectl exec $ETCD_POD -n kube-system -- \
  etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --write-out=table
```

Identify the leader:

```
kubectl exec $ETCD_POD -n kube-system -- \
  etcdctl endpoint status \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --write-out=table
```

The `IS LEADER` column shows the current etcd leader.

15.5 Lab 16.4 — Control Plane Failover Test

This test shows that workloads continue running even if the primary control plane is unavailable.

15.5.1 Deploy a workload

```
kubectl create deployment failover-test --image=nginx --replicas=4
kubectl get pods -l app=failover-test -o wide
```

15.5.2 Simulate a control plane failure

Stop the kubelet on the primary controller:

```
# On the controller node:
sudo systemctl stop kubelet
```

From another machine (or `controller2`), verify the workload still runs:

```
kubectl get pods -l app=failover-test -o wide
kubectl get nodes
```

The pods should still be `Running`. The `controller` node will show `NotReady` after the node lease expires (~40 seconds).

15.5.3 Restore the controller

```
# On the controller node:
sudo systemctl start kubelet
```

Verify it rejoins:

```
kubectl get nodes -w
```

15.5.4 Clean up

```
kubectl delete deployment failover-test
```

15.6 Lab 16.5 — kube-vip (Optional)

For a production-grade virtual IP (VIP) approach instead of HAProxy, `kube-vip` provides a Layer 2 VIP for the control plane endpoint. It runs as a DaemonSet on control plane nodes and elects a leader that holds the VIP.

```
# Generate kube-vip manifest (example — adjust VIP and interface to your environment)
VIP=<your-vip-address>
IFACE=eth0

kubectl apply -f https://kube-vip.io/manifests/rbac.yaml

docker run --rm ghcr.io/kube-vip/kube-vip:latest manifest daemonset \
  --interface $IFACE \
  --address $VIP \
  --inCluster \
  --taint \
  --controlplane \
  --services \
  --arp \
  --leaderElection | kubectl apply -f -
```

This is advanced and environment-specific — refer to the kube-vip documentation for your network setup.